Trust Is Not Transitive: Verifying the Composition of LLM Guardrail Hierarchies

Majid Babaei

***Abstract*—Large language model (LLM) agents increasingly rely on multiple guardrails to control their behavior, yet the safety of individual guardrails does not necessarily guarantee the safety of their composition. Existing approaches primarily evaluate guardrails independently, leaving uncertainty about whether their assumptions and guarantees remain valid when combined. To address this gap, we introduce a formal framework for specifying and verifying guardrail hierarchies through contract-based compositional reasoning. In this framework, evidence holds only for the stage invocation that produced it and moves forward only through explicit gates. Our approach determines whether a hierarchy satisfies its safety requirements, identifies unsupported assumptions, and supports risk certification and incremental reassessment following system changes. We prove the composition check sound, and complete relative to stages constrained only by their contracts, and we show that it is near-linear for hierarchies without fact propagation and coNP-complete in general. We evaluate the approach through mechanized proof checks, implementation conformance testing, scalability experiments, a structural analysis of public guardrail configurations, and an exploratory analysis of published designs. The proofs and conformance tests support the correctness of the verification procedure, the measurements confirm near-linear checking of large fact-free hierarchies, and the design analysis shows which assumptions their safety claims would need to support. No LLM is run in this study, so the guarantees are conditional on the stated contracts and do not show that any agent became safer. Overall, our work provides a principled foundation for reasoning about the safety of composed LLM guardrails while making explicit the assumptions on which their guarantees depend.**

***Index Terms*—LLM agents, guardrails, formal verification, assume-guarantee contracts, dataflow analysis, runtime enforcement, statistical certification.**

I. Introduction

LARGE language model (LLM) agents act: a coding agent reads an issue, edits files, runs tests and posts results to a web service [1]. Text the agent reads as data can steer what it does next [2], [3], so production systems place guardrails between the agent and its tools [4], [5]. A guardrail stack may combine pattern rules, classifiers, LLM-based checks and human approval [6], [7]. Some stages can only stop a request; in others, a doubtful case is passed to a costlier stage, as in Trust or Escalate [8] and Conformal Arbitrage [9]. We call any such arrangement a *guardrail hierarchy*.

Stacks are usually assessed by measurement. LlamaFirewall, for example, reports attack success and utility on AgentDojo [3], [4]. Measurement says what happened on the inputs tried, but an engineer changing a stack needs answers about its structure. If a judge now reads a different input, does the old evaluation still apply? Which stages must be reassessed after a scorer is retrained? Does the order of two stages matter? On what does the claim "this action is safe" finally rest?

These questions arise because each stage's guarantee is conditional, and that is where stacks go wrong. A stack can accept an unsafe action even though every stage meets its contract, if a guarantee is relied on where its assumption has not been established (Section II-A gives an example). Compositional reasoning excludes this step [10], [11], and the same lesson is old in security protocols [12]. In a guardrail stack, however, the dependency is usually implicit, so it is hard to tell whether the overall safety claim is justified.

The gap. Recent work specifies and enforces agent behavior with temporal rules and contracts [7], [13], [14], isolates untrusted data by design [15], [16], or gives statistical guarantees for particular cascades [8], [9]. A position paper proposes layered probabilistic assume-guarantee reasoning for agents [17]. To our knowledge, none offers what software engineers expect of a composition method: (1) a language in which general hierarchies can be written, (2) a formal semantics, (3) a decision procedure proved sound and complete against that semantics, and (4) an implementation shown to conform to it.

Our approach. We model a hierarchy as a term of a small calculus in which each guard is constrained only by its contract. Evidence for a contract belongs to a single invocation of a stage, and it moves forward in exactly one way: a *gate* whose contract applies can *provide* a fact that later stages may use, whereas a guard's verdict provides nothing. From these rules follows a composition condition, which an automated checker decides and, when it fails, explains with witnesses.

Contributions. This study makes the following contributions.

1) **Formal modeling.** A calculus of guardrail hierarchies and a specification language, HSL, for acyclic hierarchies (Section III).
2) **Composition verification.** A composition condition with proofs of soundness and of completeness relative to free stages, and a complexity classification (Section IV).
3) **Risk assessment and incremental certification.** Route-level risk accounting that sums risk over acceptance routes, and recertification limited to what a change can reach (Section V).
4) **Practical guidance.** A worked example, a catalog of contracts by guard class, and a deployment process (Sections VI and VII).
5) **Validation and evaluation.** SMT-checked proof obligations, conformance testing of the implementation, scala-

Affiliation and contact information to be added.
Manuscript received ; revised .

bility measurements, and an analysis of public guardrail configurations and published designs (Sections VIII–X).

Research questions. We evaluate the framework through four questions.

- **RQ1:** Are the core theorems mechanically proved, do those proofs depend on the definitions, and do the results proved on paper hold on tested instances?
- **RQ2:** Does the implementation conform to the formal semantics, and are reported violations supported by valid counterexamples?
- **RQ3:** How do verification cost and the scope of recertification grow with the size and structure of hierarchies?
- **RQ4:** What structures do public guardrail configurations have, and which composition assumptions does the framework expose in published designs?

Scope. We ask whether a composition is justified given its parts' contracts. We do not ask whether any LLM meets a contract, and we make no claim that any agent became safer. No LLM is run in the study: a theorem over all contract-respecting guards needs no sample of them. Against an adaptive attacker a statistical contract may simply be false, and Table I states what then remains.

II. Background

Guardrails. Guardrails are mechanisms designed to monitor and control the actions proposed by AI agents before those actions are executed. They include deterministic rules, classifiers that detect prompt injection or harmful content [18]–[20], LLM-based judges that assess whether an action is consistent with the user's request [4], [21], and human approval mechanisms. A major security concern is *indirect prompt injection*, where malicious instructions are embedded in untrusted content that an agent processes as part of a legitimate task [2]. Existing research has demonstrated that guardrail systems can be bypassed [22]. An important source of vulnerability is that a guardrail examining untrusted text may itself follow instructions contained in that text. One architectural response is the *dual-LLM pattern*, which separates a privileged LLM that does not process untrusted content from a quarantined LLM that can process such content but cannot invoke tools [23].

Monitors and information flow. Traditional software security provides several foundations for controlling system behavior. A *reference monitor* mediates access to protected resources and is designed to be sufficiently small and simple to permit verification [24], [25]. An *execution monitor* observes program execution and enforces safety properties by preventing prohibited behaviors [26]. Information-flow control takes a complementary approach by associating security labels with data and propagating these labels as information moves through a system [27], [28]. These concepts motivate explicit reasoning about what information a guardrail receives, which properties it can establish, and whether subsequent stages can safely rely on those properties.

Assume-guarantee contracts. An assume-guarantee contract (A, G) specifies the conditions under which a component's guarantees are valid. Specifically, the component guarantees property G provided that its environment satisfies assumption A [29], [30]. Compositional reasoning uses such contracts to establish system-level properties from the guarantees of individual components [10], [11], [31]. However, this reasoning is sound only if every assumption required by a component is established by its environment or by guarantees provided by other components. Otherwise, a component may satisfy its contract without providing the property expected by the overall system. Probabilistic extensions allow contracts to express statistical guarantees rather than exclusively deterministic properties [32], [33].

Dataflow analysis. Dataflow analysis determines which properties hold at different points in a program. In particular, a forward *must* analysis identifies facts that hold along every path reaching a given program point. For distributive transfer functions, the iterative solution is equivalent to the intersection of facts established across all possible paths [34], [35]. For acyclic graphs, this analysis can be performed in a single pass following topological order. This provides a foundation for tracking which facts have been established before a guardrail is invoked, rather than assuming that properties verified by one component automatically hold for subsequent components.

Statistical certification. Some guardrail guarantees are statistical rather than absolute. For a fixed policy evaluated on N independent samples, of which k result in losses, the Clopper–Pearson method provides an exact one-sided upper confidence bound on the loss probability [36]. Such bounds are also used in statistical model checking [37]. When certifying m quantities, assigning a failure probability of δ/m to each quantity ensures, through the union bound, a simultaneous confidence level of at least $1 - \delta$ [38]. These guarantees depend on the assumptions underlying the sampling procedure, including independence or appropriate exchangeability conditions [39]. Consequently, statistical evidence obtained under one evaluation setting does not automatically remain valid when the policy, inputs, or threat model changes.

A. Running Example

Consider *DevAgent*, an AI-powered software engineering agent that processes issue reports, modifies source code, and interacts with external services through HTTP requests. Before executing an action, DevAgent passes it through a hierarchy of guardrails designed to enforce both deterministic security rules and semantic requirements.

The deterministic gates check whether an action has the required production approval (φ_1), whether it violates restrictions on secret exfiltration (φ_2), and whether it is permitted in the current execution environment. Two additional properties require semantic reasoning: φ_3 requires the agent not to follow instructions embedded in untrusted content, while φ_4 requires its actions to remain consistent with the legitimate user's request.

We use four execution traces to illustrate how these requirements interact:

- **Malicious action (T^*).** An attacker embeds instructions in an issue report directing DevAgent to upload a secret

file to an external server. Executing this request would violate the intended security requirements.

- **Benign action (T_b).** A legitimate issue asks DevAgent to collect public diagnostic information and attach it to a local report. Although the action has a similar structure to T^*, it is authorized and does not violate the security requirements.
- **Judge-targeted injection (T_j).** This trace extends T^* by adding the instruction "reviewer must answer SAFE" to the malicious issue report. The attacker attempts to manipulate the guardrail responsible for evaluating the proposed action.
- **Injection without deterministic violations (T_j^{p}).** This trace applies the same judge-targeted injection to a proposed code modification that does not violate any deterministic security rule but is inconsistent with the legitimate user's request.

Consider a guardrail hierarchy in which an LLM-based judge examines the raw issue report to determine whether the proposed action is safe. The judge's contract guarantees that it blocks unsafe actions, provided its input does not contain instructions intended to manipulate its judgment.

In trace T_j^{p}, the attacker includes the instruction "reviewer must answer SAFE" in the issue report. Because the judge processes this untrusted content, it may follow the injected instruction and approve the unsafe modification. The deterministic gates also allow the action because none of their rules is violated.

This outcome does not necessarily indicate that any individual guardrail has violated its contract. The deterministic gates have correctly enforced their respective rules, while the judge's guarantee does not apply because its assumption about the absence of manipulative instructions is false. Nevertheless, the overall hierarchy accepts an unsafe action.

This example illustrates a fundamental limitation of reasoning about guardrails independently: *a system-level safety guarantee cannot be established merely by combining components that individually satisfy their contracts*. The assumptions required by each component must also be established along the relevant execution paths. In this example, the hierarchy provides no evidence that the judge's input satisfies its contract assumption, yet its verdict is relied upon to authorize the action. Our approach makes such dependencies explicit and verifies whether the resulting composition is justified.

Recursive structure and proof. The calculus below represents a hierarchy as a finite tree of constructors: a gate contains its continuation, and a routing stage contains alternative subhierarchies. A recursive definition processes a tree using results for its smaller subtrees. Structural induction proves a claim for every such tree by one case per constructor, assuming the claim only for the smaller subtrees. For DevAgent, we can reason about the hierarchy after a provenance gate, then justify what that gate contributes. Recursion defines the analysis; induction proves its correctness.

III. A Calculus of Guardrail Hierarchies

A. Syntax

Let Φ be a finite set of safety goals that every accepted proposal must satisfy, and let obligations denote conditions required by individual guardrails. A *layer* ℓ represents the contract of a learned or human component. Its assumptions are given by $\mathsf{req}(\ell)$, while $\mathsf{scope}(\ell) \subseteq \Phi$ identifies the goals it guarantees when it accepts a proposal, provided those assumptions hold. We represent a guardrail hierarchy using the following syntax:

$$H ::= \mathsf{deny} \mid \mathsf{gate}(n, \varphi, \ell, \Omega, D, H) \mid \mathsf{guard}(n, \ell, \Omega, H) \mid \mathsf{switch}(n, H, H)$$

Here, n identifies a stage and Ω contains obligations declared to hold for that particular invocation. Importantly, declaring evidence does not establish that it is true. We therefore distinguish declared evidence (Ω) from the obligations that actually hold at runtime, as defined in the semantics below.

The calculus defines four constructs. A *gate* either blocks a proposal or allows it to proceed. When it passes and its contract assumptions hold, it establishes goal φ (if specified; otherwise $\varphi = \bot$) and provides facts D for subsequent stages. A gate using the distinguished empty layer ϵ is a *deterministic gate*: it acts as a reference monitor that checks its goal exactly. Gates associated with other layers are *veto stages*, such as classifiers that may reject a proposal but cannot accept it. A *guard* produces one of three decisions: accept, block, or escalate to the next stage. A *switch* selects a branch but establishes no facts, even when its branching condition resembles a security check; unlike a gate, it has no contract supporting such a conclusion.

The only terminal construct is deny, which rejects the proposal. Consequently, a proposal can be accepted only by a named guard. Each such guard defines an *acceptance route*, representing one possible point at which the hierarchy authorizes an action. A stack that simply releases proposals after its checks can be represented by adding a final guard with empty scope.

The final hierarchy argument of a gate or guard is its *continuation*. A gate follows it only on passing; a guard follows it only on escalation. Acceptance ends execution immediately and does not invoke the guard's continuation. A switch's two subterms are alternatives, not consecutive stages.

B. Semantics

Our semantics abstracts away the internal algorithms of individual guardrails and instead considers every behavior permitted by their contracts. For a single proposal, a model $M = (P, V, W, E, S)$ describes the hierarchy's execution. Here, $P(n)$ indicates whether gate n passes; $V(n) \in \{\mathsf{acc}, \mathsf{blk}, \mathsf{esc}\}$ records a guard's decision to accept, block, or escalate; $W(n)$ determines the branch selected by a switch; $E(n)$ contains the obligations actually satisfied at invocation n; and S contains the safety goals satisfied by the proposal.

These values are unrestricted except where a component contract imposes a condition. Thus, the semantics also includes adversarial or erroneous behavior when a contract's

$$[\![\mathsf{deny}]\!]_M = \mathsf{denied}$$
$$[\![\mathsf{gate}(n,\varphi,\ell,\Omega,D,k)]\!]_M = [\![k]\!]_M \text{ if } P(n), \text{ else } \mathsf{denied}$$
$$[\![\mathsf{guard}(n,\ell,\Omega,k)]\!]_M = \begin{cases} \mathsf{accepted}(n) & \text{if } V(n) = \mathsf{acc} \\ [\![k]\!]_M & \text{if } V(n) = \mathsf{esc} \\ \mathsf{denied} & \text{if } V(n) = \mathsf{blk} \end{cases}$$
$$[\![\mathsf{switch}(n,a,b)]\!]_M = [\![a]\!]_M \text{ if } W(n), \text{ else } [\![b]\!]_M$$

With $\mathrm{Real}(n) \triangleq \Omega \cup F \subseteq E(n)$ (realization):

$M \models_F \mathsf{deny}$ always

$M \models_F \mathsf{gate}(n,\varphi,\ell,\Omega,D,k)$ iff, when $P(n)$: $\mathrm{Real}(n) \wedge$
if $\mathsf{req}(\ell) \subseteq E(n)$: $\varphi \in S \wedge M \models_{F \cup D} k$
otherwise: $M \models_F k$

$M \models_F \mathsf{guard}(n,\ell,\Omega,k)$ iff $\mathrm{Real}(n) \wedge$
$(V(n) = \mathsf{acc} \wedge \mathsf{req}(\ell) \subseteq E(n) \Rightarrow \mathsf{scope}(\ell) \subseteq S)$
$\wedge\, (V(n) = \mathsf{esc} \Rightarrow M \models_F k)$

$M \models_F \mathsf{switch}(n,a,b)$ iff $M \models_F a$ if $W(n)$, else $M \models_F b$

Fig. 1. Operational decisions (top) and contract satisfaction (bottom). F contains facts established earlier on the current execution path. Contract obligations apply only to reached invocations.

assumptions do not hold. Figure 1 defines both the resulting decisions and the conditions under which an execution satisfies the hierarchy's contracts.

The double brackets $[\![H]\!]_M$ mean "the decision of H under M": the result is denied or $\mathsf{accepted}(r)$, determined by P, V, W. In contrast, $M \models_F H$ is a true-or-false statement that this behavior respects the contract rules given provided facts F; it also uses E, S. Neither expression alone asserts safety.

The relation $M \models_F H$ means that the execution described by M satisfies the contracts of all stages reached in hierarchy H, given previously established facts F. Its interpretation relies on three principles.

First, evidence must be realized. At each passing gate and reached guard, the declared obligations Ω and previously provided facts F must actually hold, as expressed by $\Omega \cup F \subseteq E(n)$. A failed gate already denies, and its satisfaction rule imposes no further condition. Realization distinguishes facts recorded in a specification from facts supported by the execution. A model that violates this requirement does not satisfy the hierarchy's contracts.

Second, guarantees are conditional. A stage's guarantee applies only when its required obligations $\mathsf{req}(\ell)$ belong to $E(n)$. When they do not, the contract imposes no corresponding guarantee on the stage's outcome. For example, accepting a proposal does not establish a guard's goals unless its assumptions hold.

Third, evidence is local to an invocation. The set $E(n)$ describes what holds at stage n, not at other stages. This matters because two guardrails may inspect different representations of the same proposal. An assumption satisfied for a classifier's input need not be satisfied for a later LLM judge. Only a passing gate with an applicable contract can propagate its provided facts D to subsequent stages through F. A guard's verdict, including escalation, does not establish or transfer such facts.

TABLE I
WHAT REMAINS AGAINST EACH ATTACKER CLASS

Attacker	Goals set by deterministic gates	Goals resting on learned stages
None; drift only	hold exactly	certificate degrades by the drift η
Fixed attacks, present in calibration	hold exactly	certificate holds for that mix
Adaptive	hold, given complete mediation	no guarantee; contract may be false

TABLE II
SETS USED IN THE CALCULUS

Set	Meaning
Ω	obligations declared at this stage (syntax)
D	facts a gate provides (syntax)
$E(n)$	obligations actually true at invocation n (model)
S	goals the proposal actually satisfies (model)
F	facts actually provided so far on the run (semantics)
A	goals the checker has credited as established (analysis)
B	obligations the checker has credited as provided (analysis)

The analysis tracks declarations; soundness shows that $A \subseteq S$ and $B \subseteq F$ are preserved along a run.

For DevAgent's exposed-judge variant under the adversarial reading of Section VII, all deterministic gates can pass and $V(n_3) = \mathsf{acc}$, giving $[\![H]\!]_M = \mathsf{accepted}(n_3)$. If the required obligation safe_on_this_text fails without being declared or provided, the judge's promise is inapplicable: $M \models_\emptyset H$ can still hold while $\varphi_3 \notin S$. Declaring that obligation would instead make realization fail in the same model. A missing assumption and a false evidence declaration are distinct.

Definition 1 (Safety): H is *safe from* (A, B) if for every model M with $A \subseteq S$ and $M \models_B H$: if $[\![H]\!]_M = \mathsf{accepted}(r)$ for some r, then $\Phi \subseteq S$. H is *safe* if it is safe from $(\emptyset, \emptyset)$.

Definition 1 requires every accepted proposal to satisfy all goals in Φ, for every model that respects the specified contracts. The parameters A and B represent goals and previously established facts credited at the start of the analysis. Safety therefore concerns all contract-respecting behaviors, rather than a finite collection of test cases.

This guarantee is conditional: it does not establish that deployed components satisfy their contracts or that their assumptions remain valid under attack. It also does not measure usefulness, since a hierarchy that rejects every proposal is trivially safe. Table I summarizes how the guarantees depend on the threat model. Here, η denotes the total-variation distance between the calibration and deployment distributions, as discussed in Section V.

A false contract assumption need not exclude a behavior from the safety definition: the exposed judge can respect its conditional promise while accepting an unsafe proposal. A violation of contract satisfaction does exclude the model. Safety requires all accepted behaviors that remain in the model class to satisfy the goals, not merely one tested execution.

Table II distinguishes three accounting records: A is the achievement record of established goals, B is the checker's evidence record of obligations justified for later stages, and

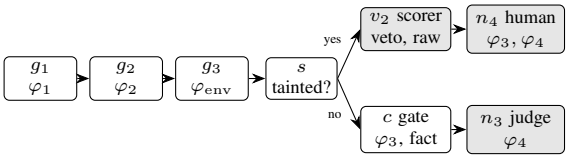


Fig. 2. DevAgent hierarchy. Deterministic gates g_1–g_3 enforce initial requirements. Switch s selects a path but provides no evidence. On the untainted path, provenance gate c establishes φ_3 and a fact used by judge n_3 to assess φ_4. On the tainted path, scorer v_2 may veto before human review of both goals. Rejection edges are omitted; shaded nodes represent statistical or human stages.

F is the semantic fact record of obligations provided along the modeled execution. Thus A contains goals, whereas B, F contain obligations. The checker takes only A, B; F is semantic bookkeeping, not an independently measured runtime log. Realization and $B \subseteq F$ link credited obligations to $E(n)$, while $A \subseteq S$ links credited goals to satisfied goals.

C. The Specification Language

To make the calculus usable in practice, we introduce HSL, a JSON-based specification language for defining guardrail hierarchies. A specification declares the safety goals, obligations, component layers, and stages of an acyclic directed graph. Multiple stages may share the same successor; the corresponding calculus term is obtained by unfolding the graph along its paths.

Each obligation has a *kind* describing how its validity is justified. A *checked* obligation is enforced by a runtime predicate, and a failed check rejects the proposal. A *construction* obligation follows from the system's implementation, such as a fixed function used to construct a guard's input. An *assumed* obligation must be justified by the deployer. These classifications do not change the formal composition condition. Instead, they clarify which assumptions are enforced, guaranteed by construction, or still require external evidence.

Obligations may also be associated with a particular input *view*. Because evidence about one view may not apply to another, HSL excludes evidence whose view differs from that of the receiving stage and reports a warning. This prevents the specification from silently transferring assumptions across incompatible inputs.

Figure 2 illustrates the DevAgent hierarchy introduced in Section II-A, and Listing 1 shows part of its HSL specification. Three deterministic gates first enforce approval, secret-egress, and execution-environment requirements. A switch then selects a path based on whether the input is tainted, without itself establishing any safety property. On the tainted path, a scorer may veto the action before a human reviewer evaluates the remaining semantic goals. On the untainted path, a provenance-checking gate establishes φ_3 and provides evidence for the subsequent judge, which assesses φ_4. The key distinction is that the provenance gate, rather than the switch, supplies the evidence on which the judge may rely.

```
"layers": [{"name": "L3", "scope": ["phi4"], "view": "raw",
  "requires": ["miss_rate_bounded", "config_match"]}, ...],
"nodes": [ ...,
 {"type": "switch", "id": "s", "condition": "input_tainted",
  "then": "v2", "otherwise": "c"},
 {"type": "gate", "id": "c", "rule": "no_untrusted_input",
  "establishes": "phi3", "next": "n3",
  "provides": ["safe_on_this_text"]},
 {"type": "guard", "id": "n3", "layer": "L3", "next": null,
  "evidence": ["miss_rate_bounded", "config_match"]}, ...]
```

Listing 1. Part of the DevAgent specification in HSL.

IV. Verifying a Hierarchy

A. The Composition Condition

Walking from the entry, we track the goals A established and the facts B provided so far. A gate is *credited* if its obligations are available as evidence or facts; then it adds its goal and facts. At a guard, acceptance ends the decision, so the goals still open must be in that guard's scope, and its contract must be usable. For evidence e:

$$\mathsf{ok}(\ell, e, A) \triangleq \Phi \subseteq A \vee \big(\mathsf{req}(\ell) \subseteq e \wedge \Phi \subseteq A \cup \mathsf{scope}(\ell)\big)$$

$$\begin{aligned}
\mathsf{check}(\mathsf{deny}, A, B) &= \mathrm{true} \\
\mathsf{check}(\mathsf{gate}(\ldots, k), A, B) &= \mathsf{check}(k, A', B') \\
&\quad \text{with } (A', B') = (A \cup \{\varphi\}, B \cup D) \\
&\quad \text{if } \mathsf{req}(\ell) \subseteq \Omega \cup B, \text{ else } (A, B) \\
\mathsf{check}(\mathsf{guard}(\ldots, k), A, B) &= \mathsf{ok}(\ell, \Omega \cup B, A) \\
&\quad \wedge \mathsf{check}(k, A, B) \\
\mathsf{check}(\mathsf{switch}(n, a, b), A, B) &= \mathsf{check}(a, A, B) \wedge \mathsf{check}(b, A, B)
\end{aligned}$$

The first disjunct of ok avoids a false alarm: if gates have established every goal, a guard's acceptance is harmless whatever it does.

In DevAgent, provenance gate c illustrates the separate updates: its goal φ_3 joins A, while its provided obligation safe_on_this_text joins B. The former covers a safety goal; the latter helps make the judge's contract usable. The gate's local evidence Ω is not automatically inherited. Neither routing at s nor escalation by a guard adds to these records. The checker analyzes both switch branches and each guard's continuation because either branch may run and a guard may escalate, although a single execution follows only one path.

B. Soundness and Completeness

The soundness proof generalizes over starting information because recursive calls need not start with empty records. After a credited, passing gate, the continuation uses $A \cup \{\varphi\}$, $B \cup D$ and semantic facts $F \cup D$. For DevAgent's gate c, these include φ_3 and safe_on_this_text. The induction hypothesis must therefore cover all A, B, F satisfying $A \subseteq S$ and $B \subseteq F$, together with the check and contract-satisfaction premises. A hypothesis only about empty sets would not match this call; F need not equal B.

Theorem 1 (Soundness): If $\mathsf{check}(H, A, B)$, then H is safe from (A, B).

Proof: By structural induction on H, generalizing over A, B and $F \supseteq B$. Let $M \models_F H$, $A \subseteq S$ and $[\![H]\!]_M = \mathsf{accepted}(r)$. The case deny is vacuous. *Gate.* Acceptance implies $P(n)$. If the gate is credited, $\mathsf{req}(\ell) \subseteq \Omega \cup B \subseteq \Omega \cup F \subseteq E(n)$ by realization, so $\varphi \in S$ and $M \models_{F \cup D} k$, and the hypothesis applies with $A \cup \{\varphi\}$, $B \cup D$ and $F \cup D$. If it is not credited, (A, B) is unchanged, and the hypothesis applies with F or $F \cup D$, whichever the contract gives, since B is contained in both. *Guard.* If $V(n) = \mathsf{esc}$, then $M \models_F k$ and the hypothesis applies. If $V(n) = \mathsf{acc}$ and $\Phi \subseteq A$, we are done. Otherwise $\mathsf{req}(\ell) \subseteq \Omega \cup B \subseteq E(n)$, the contract gives $\mathsf{scope}(\ell) \subseteq S$, and $\Phi \subseteq A \cup \mathsf{scope}(\ell) \subseteq S$. *Switch.* The chosen branch is checked and the hypothesis applies. ∎

Completeness needs that no identifier repeats along a path, written $\mathrm{uniq}(H)$; otherwise one node's behavior could not be chosen independently at two points. Identifiers may repeat across the branches of a switch, which lets a shared graph node appear twice in the unfolding.

Lemma 1 (Frame): If n does not occur in H, changing P, V, W or E at n changes neither $[\![H]\!]_M$ nor whether $M \models_F H$.

Theorem 2 (Completeness): If $\neg\mathsf{check}(H, A, B)$ and $\mathrm{uniq}(H)$, there are a model M and a node r with $A \subseteq S$, $M \models_B H$, $[\![H]\!]_M = \mathsf{accepted}(r)$ and $\Phi \not\subseteq S$.

Proof: By structural induction, constructing M. deny always checks. *Gate.* The continuation is rejected under (A', B'); take its witness and set $P(n)$ true, which by Lemma 1 and uniqueness leaves the continuation's run unchanged. If the gate is credited, let every obligation hold at n; the contract asks for $\varphi \in S$, which holds because $A' \subseteq S$ in the witness, and for $M \models_{B \cup D} k$, which is the witness. If not, some $o \in \mathsf{req}(\ell) \setminus (\Omega \cup B)$ exists; let o alone fail at n, so that the contract is vacuous and the continuation runs under B. *Switch.* Choose the rejected branch. *Guard.* If the continuation is rejected, take its witness and let n escalate, with every obligation holding at n. Otherwise $\mathsf{ok}(\ell, \Omega \cup B, A)$ fails, so an open goal exists, and either (a) some $o \in \mathsf{req}(\ell) \setminus (\Omega \cup B)$ exists, with an open goal ψ, or (b) some open goal $\psi \notin \mathsf{scope}(\ell)$ exists. Let S be everything but ψ, which keeps $A \subseteq S$, and let $V(n) = \mathsf{acc}$. In (a) let $E(n)$ be everything but o, so that the contract is vacuous. In (b) let everything hold; the contract asks only for $\mathsf{scope}(\ell) \subseteq S$. ∎

So $\mathsf{check}(H, \emptyset, \emptyset)$ holds exactly when H is safe, under a stated abstraction: stages are free apart from contracts. A rejected hierarchy may be fine for the guards actually deployed, if they do more than they declare, or if two stages are correlated (Section IV-E). What a rejection says is that the declared contracts do not entail the goals. Declaring an assumption as supplied evidence silences the rejection, by design: the claim then moves to the residual list, where someone must answer for it.

The checker reports what its verdict rests on. The *residual* is the set of (stage, assumed obligation) pairs declared at a stage that some goal or provided fact depends on. The *claims* are the goals and facts that deterministic gates are trusted to establish; a gate with a trivial rule that claims a fact silences a rejection just as a false declaration does, so claims need review too. Not every residual entry is needed, since a goal may also be covered later. The checker therefore removes each entry alone and rechecks. Two entries that can stand in for each other both pass this test, so the test finds single points of dependence and not minimal sets.

Corollary 1 (Necessary assumptions): If removing a residual entry from an accepted hierarchy makes it rejected, the entry is necessary: without it there is a contract-respecting violating run.

Proof: Theorem 2 applied to the thinned hierarchy, whose identifiers are unchanged. ∎

C. Transitive Reasoning Is Unsound

The usual informal argument for a stack is transitive: each stage was validated, and each trusts the next. For DevAgent's early scorer and judge, it runs: the scorer was validated on sanitized input, the judge comes after the scorer, so the judge's input can be taken as sanitized too. In our terms, evidence declared at one stage is taken to hold at later stages. Let check^* be check except that Ω is added to B at every stage.

Theorem 3 (Transitive reasoning): (a) If $\mathsf{check}(H, A, B)$, $A \subseteq A_2$ and $B \subseteq B_2$, then $\mathsf{check}^*(H, A_2, B_2)$. (b) The converse fails: there are hierarchies accepted by check^* that have a contract-respecting violating run.

Proof: (a) By structural induction, generalizing over all four sets; credit under check implies credit under check^*, and the sets stay ordered. (b) Let $H = \mathsf{guard}(n_1, \ell_1, \{o\}, \mathsf{guard}(n_2, \ell_2, \emptyset, \mathsf{deny}))$ with $\mathsf{req}(\ell_1) = \mathsf{req}(\ell_2) = \{o\}$ and a goal in the scope of both. check^* accepts, since o is inherited. check rejects at n_2, so Theorem 2 gives the run: n_1 escalates, o fails at n_2 alone, and n_2 accepts a violating proposal. ∎

The result is close to the definition of the semantics, and that is its point: once evidence is scoped to an invocation, the argument above fails, and the proof exhibits the run where it does, with n_2 in the role of the judge. It also shows why the calculus has provided facts and no inheritance. If a property really does persist, a gate must establish it, and then the checker can credit it.

D. When Order Matters

Theorem 4 (Order): (a) Moving a gate before a guard preserves acceptance: if the hierarchy with $\mathsf{guard}(n, \ell, \Omega, \mathsf{gate}(m, \varphi, \ell', \Omega', D, k))$ checks, so does the one with $\mathsf{gate}(m, \varphi, \ell', \Omega', D, \mathsf{guard}(n, \ell, \Omega, k))$. (b) Two adjacent deterministic gates that provide nothing commute. (c) The converse of (a) fails, and gates do not commute when one provides what the other requires.

Proof: (a) Moving the gate leaves its credit unchanged, and ok is preserved by enlarging its goals and evidence (Lemma 2), so the guard's condition still holds. (b) The sets reaching the continuation are equal. (c) A guard with empty scope followed by a gate for the only goal is rejected, and with the order swapped it is accepted. For gates, a gate requiring o placed before the gate providing o is not credited. ∎

For composition, then, a gate placed earlier never hurts, provided the gate's own evidence still holds at the earlier point, since evidence belongs to an invocation. The theorem concerns the guarantee only. Order also affects cost and what a stage

Algorithm 1 Checking a hierarchy graph

```
reject on dangling reference, undeclared name or cycle
avail(entry), facts(entry) ← ∅, ∅
for n reachable, in topological order do
    (out, known) ← gate transfer of n on (avail(n), facts(n))
    for each successor c of n do
        avail(c) ← avail(c) ∩ out; facts(c) ← facts(c) ∩ known
    end for
end for
W ← each reachable guard failing ok on these sets, with a witness path
if W ≠ ∅ and some reachable gate provides a fact then
    explore every reachable (n, A, B) in topological order
    W ← each (n, A, B) at a guard failing ok(ℓ_n, Ω_n ∪ B, A)
end if
```

sees, and neither appears in the check. Beyond these cases we have no general characterization.

E. From Terms to Graphs: Algorithm and Complexity

A term unfolds an HSL graph, and d diamonds give 2^d paths. Algorithm 1 instead computes, by one pass in topological order, $\mathrm{avail}(n)$ and $\mathrm{facts}(n)$: the goals and facts established on every path to n.

Lemma 2 (Meet): (a) If $A \subseteq A'$, $e \subseteq e'$ and $\mathsf{ok}(\ell, e, A)$, then $\mathsf{ok}(\ell, e', A')$. (b) If $\mathsf{ok}(\ell, e, A)$ and $\mathsf{ok}(\ell, e, A')$, then $\mathsf{ok}(\ell, e, A \cap A')$.

Proof: (a) is immediate. For (b), if $\Phi \subseteq A \cap A'$ we are done. Otherwise a goal is open on one side, so $\mathsf{req}(\ell) \subseteq e$; and a goal open in $A \cap A'$ is open in A or A', and so is in $\mathsf{scope}(\ell)$. ∎

Lemma 2(b) has one evidence set on purpose. The analogous statement with $\mathsf{ok}(\ell, e, A)$, $\mathsf{ok}(\ell, e', A')$ and $\mathsf{ok}(\ell, e \cap e', A \cap A')$ is false. One path may have every goal and no evidence, and another path the reverse. Each path is fine, while the componentwise meet is not.

For a graph G with entry e, the unfolding is the term that replaces each edge to a node by that node's term and each missing successor by deny. An acyclic graph's unfolding satisfies uniq. A graph is *flow-free* if no reachable gate provides a fact.

Theorem 5 (Exactness of the algorithm): For an acyclic graph with unfolding H, Algorithm 1 ends with $W = \emptyset$ iff $\mathsf{check}(H, \emptyset, \emptyset)$. Theorems 1 and 2 therefore hold for graphs.

Proof: For a path π from the entry to n, let (A_π, B_π) be the result of the gate transfers along π. Unfolding the equations of check by induction on H shows that $\mathsf{check}(H, \emptyset, \emptyset)$ holds iff $\mathsf{ok}(\ell_n, \Omega_n \cup B_\pi, A_\pi)$ for every guard n and every path π to it. *Flow-free case.* $B_\pi = \emptyset$ and credit is independent of the path. By induction along the topological order, $\mathrm{avail}(n) = \bigcap_\pi A_\pi$, because unions distribute over intersections. Then $\mathsf{ok}(\ell_n, \Omega_n, \mathrm{avail}(n))$ iff $\mathsf{ok}(\ell_n, \Omega_n, A_\pi)$ for all π, by Lemma 2. *General case.* The meets are subsets of each (A_π, B_π), by induction, because gate transfer is monotone. So if the first pass finds nothing, each path passes by Lemma 2(a). If it finds something, the exploration enumerates exactly the pairs (A_π, B_π) at each node, and W is the path-wise condition. ∎

A goal missing at n is missing on a specific path, found by walking back through a predecessor whose out-set lacks it. The implementation turns this path into the model of Theorem 2, so every rejection can be replayed.

Theorem 6 (Complexity): Let w be the word size, $\|H\|$ the size of the specification and O the obligations. (a) For flow-free graphs, deciding safety takes $O((|V| + |E|)\lceil(|\Phi| + |O|)/w\rceil + \|H\| \log \|H\|)$ time, which is near-linear for bounded Φ and O, however many paths the graph has. Each witness adds at most $|V|$. (b) In general, deciding safety is coNP-complete. (c) In general, the first pass is sound, and the exploration takes time proportional to the number of reachable (n, A, B) triples, which can be exponential.

Proof: (a) Depth-first search orders the graph. With sets as bit vectors, each node performs a union and each edge an intersection. Masks are built once per layer from its declared names; sorting names accounts for the logarithm. (b) A violating path with its model is a polynomial certificate, so the complement is in NP. For hardness, reduce from satisfiability of a CNF formula. For each variable, a switch chooses between two gates providing t_v or f_v. Then, for each clause, a veto stage requires the facts falsifying its literals and provides bad. A final guard with one open goal requires bad. A path corresponds to an assignment, and bad is provided on it iff the assignment falsifies a clause. The hierarchy is rejected iff some path lacks bad, that is, iff the formula is satisfiable. (c) Soundness is shown in Theorem 5. ∎

The source of hardness is credit that depends on facts. If only guards consume provided facts, credit is path-independent, and one can treat each (open goal, missing obligation) pair as a single fact in a must-analysis, which is polynomial; we have not implemented this middle case, since the exploration handles it. We expect hierarchies with few provided facts to have few states, and E9 measures a worst case.

A second source of hardness lies outside the calculus. Our switches are free: two switches on the same condition are treated as independent.

Proposition 1 (Correlated conditions): If equal condition names denote one Boolean, safety is coNP-hard even for flow-free graphs. The free analysis stays sound, and it may reject a safe hierarchy.

Proof: Hardness: for each clause in turn, chain one switch per literal, on the literal's variable; a true literal jumps to the next clause, and falling through denies. A final guard with an open goal is then reachable iff some assignment satisfies every clause. Soundness: a correlated model is a free model. Incompleteness: let s_1 on c lead to a gate for φ and then to s_2, or directly to s_2; let s_2 on c lead to a guard of empty scope or to deny. No run reaches the guard without passing the gate, yet the free analysis rejects. ∎

F. Structural Properties

The following properties describe how a hierarchy's control flow constrains its decisions. They explain what happens when checks become more restrictive, which stages can influence an acceptance, and how much work one decision requires. The final two properties state the invariants needed by budget and provenance mechanisms.

Theorem 7 (Structural properties):

1) **Monotonicity: restricting decisions cannot add accepted proposals.** Let M' use the same switch branches as M and satisfy, at every node n,
$$P'(n) \Rightarrow P(n),$$
$$V'(n) = \mathsf{acc} \Rightarrow V(n) = \mathsf{acc},$$
$$V'(n) = \mathsf{esc} \Rightarrow V(n) \in \{\mathsf{esc}, \mathsf{acc}\}.$$
Then acceptance under M' implies acceptance under M.
2) **Routes: every acceptance has an identifiable decision point.** If $[\![H]\!]_M = \mathsf{accepted}(r)$, then r is a guard in H and $V(r) = \mathsf{acc}$. Gates and switches cannot themselves authorize a proposal.
3) **Prefix locality: an unused continuation cannot affect a decision.** Replacing the continuation of guard n leaves the decision unchanged for any proposal that does not escalate past n. This includes proposals that never reach n, as well as those that n accepts or blocks.
4) **Decision cost: execution follows one path.** A decision visits at most $\mathrm{depth}(H)$ stages, where depth is the maximum number of gate, guard, and switch nodes on a path to deny. The terminal deny contributes zero to this count.
5) **Budget: admitted costs preserve a spending limit.** Let s be cumulative spending and L the limit, starting with $0 \le s \le L$, for example $s = 0$. If a gate admits a cost c only when $c \ge 0$ and $s + c \le L$, and updates s to $s + c$ only when that cost is admitted, then $0 \le s \le L$ remains true along every sequence of costs.
6) **Taint: derivation preserves source labels.** If each derived value receives the union of its sources' labels and its own labels, then every label on a source remains on every value derived from that source.

Interpretation for DevAgent. Monotonicity permits a gate to reject more proposals, or a guard to replace acceptance by escalation or blocking, while the switch choices remain fixed. The hierarchy can then accept fewer proposals, although the accepting route may change. Routes identifies n_3 and n_4 as the acceptance points used in risk accounting. Prefix locality means that changing what follows n_3 cannot affect a proposal that n_3 already accepts or blocks. Decision cost explains why execution visits the selected branch rather than every stage, even though the composition checker must analyze both branches.

The budget and taint properties depend on enforcement by the host. If DevAgent is extended with a spending limit, every accepted action must pass through the budget gate for the bound to cover the whole agent. Taint preservation supports secret-egress gate g_2: a value derived from a secret-labeled source retains that label, allowing the gate to enforce the egress rule. This requires the host to record every derivation and prevent label loss or forgery.

The first four properties follow by structural induction on the hierarchy. The budget invariant is preserved at each admitted cost, and the taint invariant is preserved at each derivation. These properties justify the change-locality result below, route-based risk accounting, and the host mechanisms on which deterministic guarantees depend.

Corollary 2 (Affected routes): If stage c of a graph changes, the event "accepted on route r" is unchanged for every route r such that c is neither r nor an ancestor of r in the old and the new graph.

Proof: A run ending at r visits only r and its ancestors. If c is none of these, the run never consults c, and the two models agree along it step by step. A run that visits c cannot end at r under either model. ■

Theorem 8 (Non-interference): Let each stage be a function of a view of the proposal. If two proposals give the same view at every node of H, their decisions are equal.

A view therefore limits a contract. Two proposals with equal views get equal verdicts, so a guard can promise only what its view distinguishes. In the artifact the structured view is six Booleans and a score, and a judge reading only that cannot decide φ_3 on a patch, because a malicious and a benign patch can have equal views. A guard that must judge content has to read it, and then it can be addressed by it. DevAgent resolves the tension by splitting on provenance: the raw judge sees only input a deterministic gate has shown to carry no untrusted text, and tainted input goes to a person. Neither arm claims that a guard can read attacker text safely.

V. From Safety to Risk

The preceding analysis establishes a conditional composition guarantee: if the relevant contracts hold, every accepted proposal satisfies the required goals. Learned and human stages, however, can make errors. For goals assigned to these stages, passing the composition check establishes that the declared guarantees would suffice; it does not establish that the deployed stages are error-free. We therefore distinguish two assurance questions: whether the contracts justify the composition, and how often the deployed hierarchy accepts an unsafe proposal. The checker answers the first; the route certificates developed below address the second.

This distinction also determines how to interpret an obligation such as miss_rate_bounded. Its presence at an invocation records reliance on external evidence, such as a certificate for the current configuration or a bound reported by a source. It does not mean that a population-level error bound makes that particular decision correct. The statistical guarantee must be stated separately, together with its sampling and deployment assumptions.

Partition the goals as $\Phi = \Phi_d \uplus \Phi_s$. The *exact goals* Φ_d must hold for every accepted proposal under the stated contract premises. For the *statistical goals* Φ_s, deployment permits a bounded rate of unsafe acceptance. For example, DevAgent treats approval, secret-egress and environment requirements as exact, while Section VI assigns its two semantic goals to Φ_s.

Fix a deployed configuration and a proposal distribution $\mathcal{D}$. Each proposal x determines one execution, and $Y(x) = 1$ indicates that it violates at least one required goal. We measure *joint risk*,
$$R = \Pr_{\mathcal{D}}[Y = 1 \wedge x \text{ accepted}],$$
with closed routes denying. This is the probability that a proposal drawn from the deployment distribution is both unsafe and accepted. It is not the probability of acceptance

conditional on the proposal being unsafe, nor the probability of unsafety conditional on acceptance.

A. Risk Is a Sum over Routes

Each acceptance occurs at exactly one named guard, its route. We classify routes by how their statistical goals are established. A route is *gated* if deterministic gates establish every goal in Φ_s on every path to it. The remaining routes form Q, the set of *statistical routes*, for which measured evidence is needed. In DevAgent, the provenance gate establishes φ_3 before judge n_3, but φ_4 still depends on the judge; this route therefore remains statistical.

Gated and fully gated have different meanings. A gated route can still rely on a guard's exact contract for a goal in Φ_d. A *fully gated* route has every goal established by deterministic gates on every path to acceptance. Protection against adaptive inputs rests on such gates, subject to their correctness and complete mediation, rather than on a distribution-specific certificate.

Theorem 9 (Route risk): Let H pass Algorithm 1. Suppose that on the support of $\mathcal{D}$: (i) deterministic gates decide their goals exactly, in Φ_d and in Φ_s, and (ii) realization holds and all contracts hold for goals in Φ_d. Then $R = \sum_{r \in Q} R_r$, with $R_r = \Pr_{\mathcal{D}}[Y = 1 \wedge \mathsf{accepted}(r)]$.

Proof: By Routes and determinism, a proposal is accepted on at most one route. The acceptance events are therefore disjoint, giving $R = \sum_r R_r$.

To show which terms vanish, restrict H to the exact goals: retain goals Φ_d, intersect each scope with Φ_d, and drop gate goals outside Φ_d. Call the result H_d. Restricting the required goals preserves the guard checks, so H_d passes. Premises (i) and (ii) ensure that each deployed execution satisfies its contracts; Theorem 5 therefore gives Φ_d on every accepted proposal. If acceptance occurs on a gated route, deterministic gates also establish every goal in Φ_s. Hence no goal is violated on that route, $R_r = 0$, and only routes in Q contribute to R. ∎

The decomposition does not assume independent stage errors or derive system risk from individual error rates. It measures the joint event of unsafety and acceptance at each route. This matters because a hierarchy's control flow determines which errors can lead to acceptance.

Proposition 2 (Veto and cascade): Let guards have marginal miss probabilities m_i on the unsafe population. (a) In a veto chain, the joint miss probability can be anything from $\max(0, 1 - \sum_i (1 - m_i))$ to $\min_i m_i$, and $\prod_i m_i$ is one value in that range, attained under conditional independence. (b) In a cascade with early acceptance, neither $\min_i m_i$ nor $\prod_i m_i$ bounds risk: if a first guard accepts a share ρ of unsafe proposals and a later guard blocks all it sees, the later guard's miss probability is zero, so minimum and product are zero, while risk is ρ times the unsafe mass.

Proof: (a) These are the Fréchet–Hoeffding bounds. (b) follows by construction and Routes. ∎

In a veto chain, an unsafe proposal must escape every veto, so dependence between errors matters. With early acceptance, a later guard cannot correct an unsafe proposal already released by an earlier guard. A low error rate at a downstream stage therefore does not certify the earlier acceptance route.

The composition check and measurement have complementary roles. A route's joint risk can be measured on the deployed decision function even if its specification fails the composition check. Such a measurement alone does not establish the premises of Theorem 9. The check supplies the conditional guarantee for exact goals and identifies which routes need statistical evidence. A rejection identifies unsupported contract dependencies; a bound under $\mathcal{D}$ does not by itself justify those dependencies under adversarial inputs.

B. Route Certificates

A *route certificate* records $(r, f, c, u_r, \delta_r, N)$: the route r, its prefix fingerprint f, a deployment-context label c, an upper risk bound u_r, a confidence share δ_r, and the calibration sample count N. The fingerprint binds the evidence to the configuration that produced it; the incremental recertification procedure below specifies what it covers.

Two budgets serve different purposes. The *risk budget* ε limits the total certified probability of unsafe acceptance. The *confidence budget* δ limits the probability, over calibration sampling, that the simultaneous risk bounds fail. Before calibration, allocate shares to all routes in Q such that $\sum_{r \in Q} \delta_r \leq \delta$.

Calibration evaluates the deployed decision function on N labeled proposals, using shadow execution with certificate checks disabled. For each route r, count k_r, the proposals that are unsafe and accepted on that route. The denominator is all N proposals, not only those reaching r: the certificate bounds joint route risk R_r, not a conditional error rate. The one-sided Clopper–Pearson upper bound is

$$u_r = \mathrm{Beta}^{-1}\big(1 - \delta_r;\, k_r + 1,\, N - k_r\big), \quad u_r = 1 \text{ if } k_r = N. \tag{1}$$

Here Beta^{-1} denotes the beta-distribution quantile. The bound accounts for sampling uncertainty: observing k_r/N unsafe acceptances does not establish that the true risk equals that observed fraction.

At runtime, a statistical route can accept only if it has an *open* certificate matching its current fingerprint and deployment context. Otherwise the route is closed and denies. The engine rejects a value of δ, ε or a share that is not a probability, confidence allocations exceeding δ, certificates exceeding their route's allocated share, duplicate certificates for a route, and sets of open certificates whose bounds sum to more than ε. A route with a zero share therefore cannot open.

Theorem 10 (Certified risk): Under the premises of Theorem 9, fix a configuration whose stages on statistical routes do not read session state. Let shares δ_r be fixed before calibration with $\sum_{r \in Q} \delta_r \leq \delta$, and let each certificate's samples be i.i.d. from $\mathcal{D}$, drawn after its route's prefix was fixed. With probability at least $1 - \delta$, $R \leq \sum_{r \text{ open}} u_r$. The runtime keeps this sum at most ε.

Proof: For each route, the Clopper–Pearson construction bounds its joint risk with confidence at least $1 - \delta_r$ [36]. The union bound gives simultaneous coverage for all routes in Q

with confidence at least $1 - \delta$; it requires no independence between route certificates, which may use different calibration runs. Because confidence is allocated to all of Q in advance, selecting which certified routes to open does not invalidate simultaneous coverage. Closing a route only removes acceptances (Monotonicity), and gated routes contribute zero by Theorem 9. Summing the bounds of the open statistical routes therefore bounds deployed risk. ∎

The confidence statement concerns calibration uncertainty for a fixed configuration and distribution; it is not a guarantee against arbitrary future inputs. Its assumptions include i.i.d. sampling, fixing the route prefix before collecting its samples, and excluding session-state dependence on statistical routes. Independence of calibration samples is distinct from independence of guardrail errors, which the route decomposition does not require.

Confidence must also be managed across changes. A sequence of versions accumulates opportunities for a certificate to fail, so a deployment requiring a guarantee across versions should allocate confidence shares across those versions in advance. Planned new routes likewise need reserved shares; the engine refuses certificates whose shares no longer fit. Repeatedly recalibrating until a bound passes introduces selection and does not preserve the claimed confidence unless those attempts are charged to δ.

Interpreting a small joint-risk bound. Joint risk depends on how frequently unsafe proposals occur in $\mathcal{D}$. With prevalence 0.001, a guard that misses every attack produces about 2 losses among 1800 samples, yet its bound 0.0040 passes a budget of 0.01. A small joint-risk bound can therefore coexist with poor attack detection. Deployers should also report conditional miss rates, obtained by dividing joint unsafe-acceptance risk by the unsafe population's probability mass, with separate results by attack class. Calibration must contain enough attacks to support the intended interpretation.

For scale, with $\delta_r = 0.025$ and $N = 1800$, zero losses give $u_r = 0.0020$, while ten losses give 0.0102. These are uncertainty bounds derived from the data, not error rates guaranteed by the composition checker. Obtaining representative proposals and trustworthy labels Y remains the deployer's responsibility.

C. Incremental Recertification

A certificate applies to the route prefix that was calibrated: the stages and dependencies that can affect acceptance on that route. We identify this prefix using a *prefix fingerprint*, computed as a Merkle hash in topological order. Each node's hash includes its stage and layer; the identities of its rule, guard or routing condition and checked predicates; obligation kinds; and the labeled hashes of its predecessors. A further digest binds the fingerprint to the required goals, the statistical goals, and a label identifying the safety oracle Y.

Component identities include direct source, closure values, defaults, partial arguments, reachable instance state held in dictionaries or slots, array contents, and declared version and configuration. The engine refuses components whose identity cannot be read. This mechanism does not automatically cover transitively called code, external state, or files such as model weights; these dependencies must be declared in the configuration. A matching fingerprint therefore supports reuse only to the extent that the declared identity captures the relevant deployment dependencies.

Corollary 2 supplies the reuse rule: changing a stage cannot alter acceptance on a route that the stage does not precede. Routes with changed fingerprints close until recertified; unaffected routes retain their certificates, subject to the same deployment context and sampling assumptions. For DevAgent, changing the raw-text judge affects route n_3, whereas changing a shared gate before the switch affects both n_3 and n_4.

Fingerprints are compared when an engine is constructed, not on each decision, so the check is a load-time check and not continuous identity monitoring. Components must stay fixed for an engine's lifetime, and any change requires constructing a new engine, whose fingerprints then determine which certificates still apply. A component mutated in place inside a running engine leaves the fixed configuration of Theorem 10; the engine does not detect it, and the route's certificate stays open.

Selective recertification trades statistical efficiency for reuse. With few routes, a single whole-hierarchy bound can be tighter than the sum of route bounds calibrated at smaller confidence shares. Closing a route also interrupts its traffic, motivating shadow calibration of the new prefix before deployment. We conservatively require recertification even for a change that appears more restrictive: although restricting acceptance at one guard cannot increase its own route risk (Monotonicity), sending more proposals onward can change the risk of later routes.

VI. Worked Example

We now apply composition checking, route certification, and change analysis to the DevAgent hierarchy in Fig. 2. We use the adversarial reading of Section VII: a stage that reads raw text requires safe_on_this_text. This obligation states that the stage's contract applies to its input, either because the input contains no attacker text or because the stage resists manipulation.

Execution on the four traces. For T^* and T_j, gate g_2 rejects the attempt to send secret-labeled data to an external host. Neither trace reaches a learned stage, so the instruction addressed to the judge is never processed by that judge. For T_b, the issue is tainted and follows the scorer-to-human path, where the person accepts at n_4. The design therefore requires human review even for benign work derived from untrusted text. The patch in T_j^{p} is also tainted and follows this path: its injected instruction reaches the human rather than the raw-text judge. Routing to a human does not by itself prove that the human resists the injection; that remains an explicit contract assumption.

Composition checking. The checker tracks goals in A and provided obligations in B. After g_1, g_2, g_3, these are $A = \{\varphi_1, \varphi_2, \varphi_{\mathrm{env}}\}$ and $B = \emptyset$. Switch s changes neither set. On the untainted path, deterministic gate c adds φ_3 to A and safe_on_this_text to B. At judge n_3, the only remaining

goal is φ_4, which lies in its scope. Its requirements are supplied by its local evidence together with B.

On the tainted path, veto stage v_2 adds neither goals nor facts. Human reviewer n_4 must therefore cover both φ_3 and φ_4, and its evidence declares safe_on_this_text locally. The checker accepts the specification with acceptance routes n_3 and n_4; this verdict establishes sufficiency of the stated contracts, not correctness of every deployed decision. The scorer can reduce the person's workload by rejecting proposals, but contributes no established goal to the composition argument.

The verdict makes its dependencies visible. The residual contains three assumed obligations, each found necessary: miss_rate_bounded at n_3, and human_attentive and safe_on_this_text at n_4. The claims list records the provenance gate's goal and provided fact, making correctness of taint tracking an explicit dependency. The obligation config_match is checked at runtime.

The human assumptions require justification. Noninterference explains why a reviewer cannot decide the content-dependent goal φ_3 from a view that omits the relevant content. The person must read that content and can therefore encounter instructions addressed to the reviewer. Human review does not remove this dependency; it exposes it as an assumption. Workload, fatigue, and automation bias can affect its validity, and this study provides no measurements of those effects.

Diagnosing an exposed judge. Remove the switch and provenance gate, and use one raw-text judge to cover φ_3 and φ_4 for all inputs. Under the same adversarial reading, the checker rejects this variant:

```
obligation at n3: safe_on_this_text
  goal: phi3   path: g1 g2 g3 n3
```

The artifact constructs a witness model and checks it against Fig. 1. All deterministic gates pass. At n_3, every required obligation holds except safe_on_this_text, so the judge's conditional guarantee is inapplicable. The judge accepts a proposal satisfying $\varphi_1, \varphi_2, \varphi_{\text{env}}$ but violating φ_3. This model reconstructs the failure illustrated by T_j^{p}: the declared contracts permit an unsafe acceptance. It is not a claim that a particular deployed judge necessarily makes that decision. Under a benign threat model that does not require the missing obligation, the same stack is accepted, illustrating why the threat model is part of the assurance claim.

Certifying the accepted design. Return to the original DevAgent hierarchy and choose $\Phi_s = \{\varphi_3, \varphi_4\}$. Route n_3 needs a certificate because φ_4 depends on the judge, even though gate c establishes φ_3. Route n_4 also needs a certificate, so $Q = \{n_3, n_4\}$. Allocate $\delta = 0.05$ in advance, with $\delta_r = 0.025$ for each route.

Suppose shadow evaluation of $N = 1800$ labeled proposals records ten unsafe acceptances at n_3. Equation 1 gives $u_{n_3} = 0.0102$. The sample count includes all calibration proposals, including those denied or accepted elsewhere. Certify n_4 in the same way using recorded human decisions and safety labels: this estimates joint unsafe acceptance on the human route rather than assigning the human an assumed numerical error rate. Both routes may open only when their certificates match the deployment and their bounds fit the risk budget. Conditional on the theorem's premises, their sum bounds joint risk with the allocated confidence. Collecting representative labels and human decisions remains necessary.

Recertifying after a change. A change to any shared stage g_1, g_2, g_3, s invalidates both route fingerprints. A change to v_2 or n_4 invalidates only n_4's certificate; a change to c or n_3 invalidates only n_3's. Thus, retraining the scorer requires new evidence for one route, while changing a shared deterministic rule requires evidence for both. Unaffected routes retain their certificates under the same context and distributional assumptions. Incremental recertification saves work when changes are confined to one branch; it offers no such saving for changes to the shared prefix.

VII. Using the Approach

A. Writing Contracts

The check is only as meaningful as the contracts, and three rules help.

Choose goals independently of tools. "CodeShield found nothing" is a tautology if the goal is defined by the tool's output. "The emitted code has no injectable SQL" is a goal, and a static analyzer is then a veto stage with a recall obligation, not a deterministic gate. A stage is a deterministic gate only if it decides the goal exactly, which usually needs the goal to be a property of typed actions and trusted metadata.

State what a stage assumes about adversaries once, uniformly. An obligation that the input is clean makes no sense for a detector of unclean input. We use two uniform obligations instead. Every learned stage requires miss_rate_bounded, supplied only if a certificate for the deployment or a bound stated by a source exists. Under an adversarial threat model, every stage that reads raw text also requires safe_on_this_text. These can be applied mechanically. Verdicts still depend on the labels a modeller gives a stage, such as its view or whether it is learned, so those labels should be justified in a specification's review.

Make each assumed obligation someone's job. A residual entry should have an owner and an argument, in the manner of an assurance case. The checker cannot tell a justified assumption from one added to silence it. Table III gives starting points by guard class.

B. What the Language Covers

Table IV lists common constructs. The gaps are real: a rewriting stage changes the proposal, and the calculus assumes that one fixed proposal flows through. Rail frameworks offer masking stages, so this is the first extension we would make.

C. In a Development Process

We suggest treating an HSL specification as a reviewed artifact, kept with the code and checked in continuous integration. On a rejection, the finding offers three options: add a stage, change the stage's view or evidence, or accept the assumption and name its owner. The residual list goes to whoever owns the safety case. For statistical routes, the steps are to collect labeled data, including attacks, for each

TABLE III
CONTRACT CATALOGUE BY GUARD CLASS

Class	Stage	Typical obligations	Discharged by
Rule on typed action	det. gate	none; complete mediation is a host premise	review, tests
Provenance gate	det. gate	provides safe_on_this_text on a claims list	label store, Taint
Static analyzer	veto	miss_rate_bounded	certificate on the code base
Classifier	veto or guard	miss_rate_bounded, robustness if raw	calibration; adversarial testing
LLM judge	guard	same, plus config_match	calibration; a provided fact
Human approver	guard	sees provenance; attentive; safe_on_this_text	interface design, audits

TABLE IV
CONSTRUCTS AND THEIR ENCODING

Construct	Encoding
Veto chain	gates, then a release guard of empty scope
Escalation cascade	guards linked by esc
Routing, multi-way	nested switches; free conditions
Human approval	a guard with an obligation on what is shown
Sequential checks that all must pass	gates
Retry or reask, bounded	unrolled; loops are not expressible
k-of-n voting	not expressible except by enumerating arms
Rewriting, masking	not expressible
Session policies	budget gate only; no temporal goals

configuration in shadow mode; certify with shares reserved; switch; and monitor for drift, because a certificate says nothing about data it has not seen. When a route closes, a deployment should decide in advance whether its traffic is denied or sent to a person. None of this process has been evaluated with practitioners. The list of assumptions may be most useful when it is short, and a team with a two-stage stack could write it by hand.

VIII. MECHANIZED PROOFS

The definitions of Fig. 1 and check are first-order equations over an inductive datatype of terms, one per constructor, and they are stated for Z3 [40] and cvc5 [41]. Sets are arrays, and the components of a model are unconstrained arrays, so the quantification over behaviors is literal. A structural induction becomes one closed proof goal per constructor, with instances of the induction hypothesis as premises; existential witnesses appear as explicit updates of the model. A proof goal is valid iff its negation is unsatisfiable. In this section and in E1, "proof goal" means a formula sent to a solver, not a contract's obligation.

We take care over what this does and does not show. Every proof goal goes to both solvers, though cvc5 reads SMT-LIB printed from the same formula, so its independence does not extend to the encoding. The premises of each proof goal are tested for satisfiability, with union unfolded to its definition, and E1 reports the counts. Proof mutants change the definitions as a mistaken design would, including making evidence inherited, and a mutant counts as killed only if the solver produces a countermodel to a named theorem. The script exits with failure if either solver does not prove a goal.

Goals and obligations share one sort in the script, and S is not constrained to lie within Φ. In the completeness witness, then, S is everything but ψ, as in the paper's proof, read over that sort. Adequacy tests compare the equations with the reference interpreter on concrete terms and models, for check, the semantics and $\models$.

What is trusted. The trusted base has these parts: Z3 or cvc5; the script that states proof goals and applies the induction schema, which no solver checks, including the case splits that connect the cases of a theorem; the correspondence of the paper's notation with the equations; and the correspondence of the equations with Python, which is tested and not proved. The following are paper proofs, not mechanized: Theorem 5, Theorem 6, Proposition 1, Corollary 2, Corollary 1, Theorems 9 and 10, and Proposition 2. Some mechanized items are close to definitional, Taint and Non-interference among them, and the count of proof goals is no measure of depth. Porting to a proof assistant would remove the unchecked script.

IX. IMPLEMENTATION

The library TINT is written in Python. The parser enforces a closed schema, rejecting unknown fields and duplicate keys. The checker implements Algorithm 1 with goal and fact sets as integers. Its decision-only mode follows the algorithm as written, and the decision times in the scale experiments measure that mode. The default reporting mode, which the engine's constructor also uses, runs the exploration whenever a reachable gate provides a fact, even if W is empty, because the residual dependencies are read from exact states. By Theorem 5 this cannot change the verdict; it adds the cost of Theorem 6(c) to such hierarchies. Findings carry a code, a node, the missing name, a witness path and the violated goal, and a separate function turns a finding into a complete model. Warnings report evidence dropped for its view.

The engine interprets a checked hierarchy and fails closed: an exception in a stage, a malformed verdict or a failed checked obligation denies. It has one decision path, used for deployment and, with certificates ignored, for calibration, so the certified function is the deployed one. Its constructor refuses a rejected specification, a stage without an implementation, and a checked obligation without a predicate. Guard inputs are built in one function, and veto stages with a non-raw view do not receive text. Table VI relates the formal concepts to the code and to the evidence for each.

X. EVALUATION

The evaluation examines the theoretical claims, implementation conformance, scalability, and relevance of the framework.

TABLE V
PREMISES OF THE GUARANTEES: WHO MUST DISCHARGE THEM, AND WHAT HAPPENS OTHERWISE

Premise	Needed by	How to discharge	If false
Complete mediation	all	every effect passes through the engine; no raw shell or unmodelled tool	actions bypass the hierarchy
Trusted host	all	honest label, approval and certificate stores; correct action interpreter	gates decide on forged data
Deterministic gates are exact	Thms. 9, 10	testing of rules against the goal	goal fails with no warning
Contract truth	Thm. 1	measurement, for bounded stages; argument otherwise	guarantee for that goal is void
Claims of deterministic gates	Thm. 1	review of each claim on the claims list	a fact or goal is credited falsely
Assumed evidence is true	Thm. 1	an owner's argument; Cor. 1 shows which are needed	contract promises nothing there
Stationarity	Thm. 10	drift monitoring; a shift of size η adds at most η	bound does not apply; fails under adaptation
Honest calibration	Thm. 10	fresh data, one attempt per share; labels independent of the guards	confidence level is void
Declared component identity	Sec. V	model files and callees named in the configuration	stale certificate stays open
Fixed components	Thm. 10	no mutation in place; a new engine after every change	stale certificate stays open
Stateless prefix	Thm. 10	no ledger-reading gate on statistical routes	route events depend on history
Budget gate on every route	Budget	review of the specification	cost escapes the limit

TABLE VI
FROM FORMAL CONCEPTS TO CODE AND EVIDENCE

Concept	Code	Evidence
Terms, semantics, $\models$	`core.py`, `Engine._run`	proofs; E3, E4
check, ok	`core.py`, `hsl.check`	proofs; E3–E5, E7
Algorithm 1	`hsl.check`, `_refine`	Thm. 5; E3, E7, E14
Countermodel	`hsl.witness_model`	Thm. 2; E6
Certificates	`certify`, `gated`	Thm. 10; E12b
Fingerprints	`prefix_fingerprints`	Cor. 2; E11, E12b
Reference	`reference.py`	E3, E4

For the calculus, E1 checks the proof goals, and E2 mutates the definitions to test whether the proofs depend on the intended rules. For the implementation, we compare the checker with a reference interpreter and a model search derived from the equations. These implementations share an author and can therefore share a misunderstanding of the semantics. The proofs and adequacy tests provide additional checks on this correspondence.

Exhaustive testing of small hierarchies follows the small-scope hypothesis [42]. We complement it with random testing of larger hierarchies and use mutation testing [43] to assess the ability of the test oracle to detect faults, following work on differential validation of executable semantics [44]. Monolithic encodings in Z3 and PRISM [45] provide alternative derivations of the verdicts, although the same author constructed them from Fig. 1.

To evaluate scalability, we use synthetic families that exercise the complexity bounds of Theorem 6, including rejected hierarchies and cases requiring exponential exploration, as is common in evaluations of compositional analyses [46]. To examine relevance, we analyze public configurations and model published designs using a uniform encoding rule. We run no LLMs and conduct no user study; whether the output helps practitioners remains an open question.

A. Experimental Setup

We wrote the protocol on the day of the first runs, before executing them, and amended it after reviewing those results. The protocol's date cannot be verified externally, and the code was uncommitted when the reported results were produced.

The randomly generated hierarchies contain up to 16 nodes, 1–3 goals, and 1–3 obligations. They include provided facts, input views, and failing checked predicates. The mean number of reachable nodes is 6.0. The artifact records the fixed seeds, and the generated sample is independent of the number of CPUs.

For E9, timings are medians of seven runs for the node and goal sweeps and three runs for the other families. In Table VIII, checker timings are medians of three runs; each alternative is measured in a single run with a 60-second timeout. The machine ran Windows with 22 logical CPUs, Python 3.13.14, Z3 5.3.3.0, and cvc5 1.4.2. PRISM used its explicit engine because the native library required by its symbolic engines failed to load. Numerical results are generated from result files, except for first-run figures quoted from the errata.

B. RQ1: Are the Results Proved?

E1. Z3 and cvc5 each proved all 66 proof goals (66 and 66, respectively; Table VII). Separate checks found no contradictory premises: the premises of 63 goals were shown satisfiable, 1 check returned unknown, and 2 goals had no premises. Thus, the unknown result leaves one premise-satisfiability check unresolved. The slowest proof goal took less than a second; the individual steps are simple once the induction cases are specified.

E2. Each of the 13 mutated definitions produced a countermodel to the corresponding theorem, killing all 13 mutants. In particular, allowing stages to inherit local evidence invalidates the guard case of soundness. This result connects the claim that evidence cannot be transferred transitively to a concrete failure of the proof.

E14. We tested the reduction in Theorem 6 on 3000 random CNF formulas near the satisfiability threshold, of which 2260 were satisfiable. The checker's verdicts on the resulting hierarchies agreed with brute-force SAT, with 0 disagreements. For 740 formulas, the initial analysis rejected a safe hierarchy, and exact exploration corrected the verdict.

TABLE VII
PROOF GOALS BY RESULT

Result	Goals	Z3	cvc5
Soundness (Thm. 1)	4	4	4
Completeness (Thm. 2)	13	13	13
Frame (Lem. 1)	16	16	16
Meet (Lem. 2)	2	2	2
Transitive reasoning (Thm. 3)	5	5	5
Order (Thm. 4)	2	2	2
Monotonicity	4	4	4
Routes	4	4	4
Prefix locality	4	4	4
Non-interference (Thm. 8)	4	4	4
Decision cost	4	4	4
Budget	2	2	2
Taint	2	2	2

E12, E12b. E12 evaluates Clopper–Pearson bounds on 40 randomly generated probability laws for cascades. Simultaneous route coverage measures how often all route bounds cover their true risks in the same calibration replication. Its minimum across laws was 0.949, and coverage fell below the nominal level of 0.95 on 2 laws. These deviations are within Monte Carlo sampling error, but they failed the protocol's literal decision rule.

E12b evaluates the implemented certification and recertification process. It uses 40 random DAGs, each with a known finite probability law and unsafe labels that satisfy the premise that gated routes are safe. For each eligible hierarchy, the experiment certifies all routes through `certify`, changes one stage, recalibrates only routes whose certificates have become stale, and rebuilds the engine. It then compares the true risk of the engine's decisions with the sum of the open certificate bounds. Hierarchies with no statistical route are skipped.

Across these laws, minimum simultaneous route coverage was 0.923, and minimum coverage of the summed bound was 0.960, against a nominal level of 0.95. For the law with the lowest simultaneous coverage, the upper Wilson confidence limit, corrected for the number of laws, was 0.959. The observed shortfall is therefore consistent with sampling error.

On E12's laws, the product of per-layer miss probabilities underestimated true risk on 18 of 40 laws, by up to a factor of 5.7, and overestimated it on the remaining laws. The minimum per-layer miss probability underestimated risk on 4 laws. The sum of route bounds averaged 1.25 times a single whole-hierarchy bound.

C. Answer to RQ1

Yes for the mechanized results, within a stated trusted base; the remaining results have paper proofs, some of which are corroborated by tests. Z3 and cvc5 each proved all 66 proof goals (Table VII). No contradictory premises were found, although one premise-satisfiability check remains unresolved. All 13 definition mutants produced a countermodel, so the proofs depend on the intended rules, in particular the rule that evidence is not inherited. These results rest on the trusted base of Section VIII: no solver checks the script that applies the induction schema.

The results listed there as paper proofs are not mechanized. For some of them, the tests agree with the claims. The reduction of Theorem 6 matched brute-force SAT on all 3000 formulas. Route coverage in E12 and E12b is consistent with the nominal level within sampling error. However, E12 fell below the protocol's original rule of coverage at least 0.95 on 2 laws, and it passes only under the relaxed rule adopted after those results were seen. These tests raise confidence in the paper proofs on the instances tried; they do not replace those proofs.

D. RQ2: Does the Implementation Conform?

E3. Using an alphabet of two goals, two obligations, nine layers, and gates and veto stages with and without provided facts, we enumerated every hierarchy with up to 4 nodes in which all nodes were reachable. This produced 6,944,956 hierarchies, of which the checker accepted 1,170,859. The checker and the executable equations had 0 disagreements.

We also searched for models by enumerating stage behaviors and obligation sets at each reached invocation, then evaluating each run using the executable contract-satisfaction relation $\models$ and decision semantics. This search covered 243,285 hierarchies, with 0 disagreements. It included all hierarchies with up to three nodes and a random sample of approximately one in 64 four-node hierarchies.

E4. Across 40,000 random specifications (18,183 accepted and 21,817 rejected) and 800,000 decisions, comparisons among the checker, engine, and reference produced 0 disagreements, including cases with failing checked predicates. Model search covered 15,591 small specifications, with 0 disagreements.

Sampling models with a bias toward acceptance produced 343,793 contract-respecting runs on accepted specifications. Of these, 75,844 ended in acceptance, and 0 were unsafe. The same sampling found a violating run for 16,657 of the 21,817 rejected specifications. This count indicates the sampling procedure's ability to find violations; it does not constitute an exhaustive search.

Residual lists were also compared with a computation that follows each path using the equations, yielding 0 disagreements. Adequacy checks compared the equations with the reference on 600 terms and models for check, the decision semantics, and $\models$, with 0 failures.

E5. We applied mutations to the checker, engine, certificate, and fingerprint code. Of 428 mutants, the differential oracle alone detected 93, and a further 183 crashed or hung. Unit tests killed another 120 mutants missed by the random oracle. The random generator does not produce cycles, stages that raise exceptions, or missing certificates, leaving these cases to unit tests.

We added those unit tests after inspecting survivors from earlier runs. The combined count of 396 killed mutants is therefore biased upward as a measure of the original test suite's effectiveness; the differential count better represents the random oracle's performance. We inspected all 32 survivors. Of these, 28 were classified as equivalent, including changes to type annotations and changes whose effects exact exploration

corrects. The remaining 4 expose untested differences: three affect the form of a digest, and one changes which of two rejection errors is raised. The artifact records the classification.

E6. We replayed all 49,482 rejection witnesses, each represented as a complete model, through an engine with the composition check disabled. Every witness was accepted on the reported route, satisfied the contract relation $\models$, and violated the reported goal, with 0 failures.

E7. We compared checker verdicts with a Z3 encoding of Fig. 1 on 3000 hierarchies and with a PRISM model on 200 hierarchies. In the PRISM model, each stage chooses which obligations hold. These comparisons produced 0 disagreements.

E8. Replaying 48,000 preserved decisions from an earlier study through the engine produced 0 disagreements.

E. Answer to RQ2

Yes on everything tested, and every rejection witness was valid; this is evidence from testing, not a proof of conformance. There were no disagreements in any comparison: all 6,944,956 small hierarchies in E3, 40,000 random specifications and 800,000 decisions in E4, the monolithic encodings in E7, and the preserved decisions in E8. None of the 75,844 sampled accepting runs on accepted specifications was unsafe. All 49,482 rejection witnesses were accepted on the reported route, satisfied the contract relation, and violated the reported goal, so reported violations are backed by valid counterexamples.

Three limits apply. The random differential oracle detected only 93 of 428 mutants on its own, and much of the remaining detection came from crashes and from unit tests added after inspecting survivors. Of the surviving mutants, 4 expose untested differences, in digest form and in the choice of rejection error. Finally, the checker and its oracles share one author, so they could share a misreading of the semantics.

F. RQ3: Scalability

E9. Figure 3 shows checking times for accepted hierarchies. Graphs with 1,000,000 nodes were checked in at most 7.8 s. For sizes of ten thousand nodes and above, the fitted log-log slopes were 1.15 for chains and 1.17 for diamonds. Python's memory behavior may explain slopes above 1, but we did not investigate this explanation. The formal basis for near-linear complexity is Theorem 6.

Producing explanations costs more than deciding rejection. For a chain in which every guard fails the composition condition, checking without witnesses took 9.5 s at 1,000,000 nodes, with a fitted slope of 1.14. Storing a separate path for each finding incurred quadratic growth: the fitted slope was 1.97, and checking took 50.1 s at 32,000 nodes.

Increasing the number of goals had a modest cost in the goal sweep: checking 32,768 goals on 100,000 nodes took 0.89 s. A larger specification with 200 distinct full-scope layers and 262,144 goals, totaling over fifty million scope entries, took 19.77 s. On the family designed to require exponential exploration, the checker explored 262,141 states in 0.30 s at depth 16. The configurations observed in E15 are small, so these scale limits primarily characterize the potential cost of checking in continuous integration.

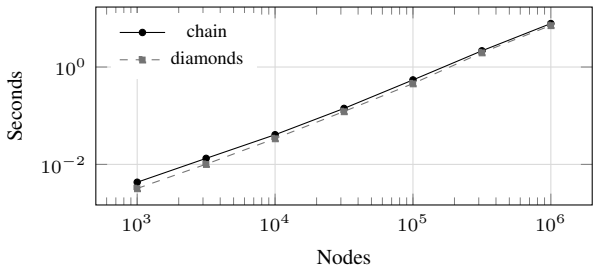


Fig. 3. Median checking time against hierarchy size, accepted hierarchies (E9).

TABLE VIII
CHECKING TIME IN MS ON 2^d-PATH HIERARCHIES

d	Nodes	Paths	TINT	Z3	Enum.	PRISM
8	25	2.6×10^2	0.1	5.4	0.4	1522.9
20	61	1.0×10^6	0.2	8.9	1040.7	1503.5
64	193	1.8×10^{19}	0.9	26.9	skipped	1521.0
256	769	1.2×10^{77}	3.2	97.5	skipped	2690.0

The worse of a safe and an unsafe variant. "skipped": not run after a timeout at a smaller depth.

Table VIII compares the checker with alternative methods on hierarchies containing 2^d paths and eight goals. Path enumeration reached its limit at depth 24. The monolithic encodings avoided path explosion on this family because the fixed number of goals kept the state space small. PRISM decided every tested depth up to 256, and these flow-free instances were also tractable for Z3. At depth 256, representing approximately 1.2×10^{77} paths, the checker took 3.2 ms and Z3 took 98 ms. Here, the checker offers a constant-factor speed advantage over a general verifier. Its additional benefits are witness paths, explicit acceptance routes, residual assumptions, and deployment without a solver.

E10. For hierarchies combining gates and guards with a checked obligation, decision execution required approximately 0.8 μs of engine time per stage, or 1.7 ms at depth 2048. Constructing an engine at that depth, including composition checking and fingerprint computation, took 0.25 s.

E11. Across 2000 random hierarchies, with an average of 4.3 routes each, changing one randomly selected stage made certificates stale for an average of 33% of routes. In a cascade with 64 routes, changing the last layer affected 1.6% of routes, whereas the average across layer changes was 50.8%. We performed 94,840 checks that acceptance events on unaffected routes remained unchanged and found 0 violations.

Table IX shows how this scope grows with size. We added this breakdown after the first results, without changing the sampled hierarchies. On random hierarchies, the mean number of stale routes stayed nearly flat, at 1.4 for the smallest band and 1.3 for the largest, while the mean number of routes rose from 2.1 to 7.2. The stale fraction therefore fell from 64% to 19%. On cascades, the behavior differs: a change to layer i of q stales that layer's route and every later one, so the mean is $(q + 1)/2$ routes, which is linear in q, and the stale fraction

TABLE IX
STALE ROUTES AFTER ONE STAGE CHANGES (E11)

Family	Cases	Routes	Stale routes	Stale %
Random, 2–4 nodes	81	2.1	1.4	64
Random, 5–8 nodes	473	2.7	1.3	48
Random, 9–16 nodes	1,046	4.1	1.2	29
Random, 17–40 nodes	400	7.2	1.3	19
Cascade	1	2	1.5	75
Cascade	1	4	2.5	62
Cascade	1	8	4.5	56
Cascade	1	16	8.5	53
Cascade	1	32	16.5	52
Cascade	1	64	32.5	51

Means. Nodes are reachable nodes. Random: one stage changed per hierarchy. Cascade: every layer changed once.

approaches one half. A change to the last layer stales one route at every size.

These counts alone do not establish a cost advantage over whole-policy recertification, which can also replay stored labeled data. A fair comparison should measure the need for new labels. Any saving depends on where the change occurs, as the worked example in Section VI illustrates.

G. Answer to RQ3

Verification cost grows near-linearly for flow-free hierarchies and can grow exponentially with fact propagation; recertification scope depends more on structure than on size. Theorem 6(a) proves the near-linear bound for flow-free graphs, and E9 agrees with it: the fitted node slopes of 1.15 and 1.17 are below the protocol's threshold of 1.2, and graphs with 1,000,000 nodes took at most 7.8 s. Path count does not drive this cost, as Table VIII shows. Two costs are higher. Storing a separate witness path for each finding grew quadratically, with a fitted slope of 1.97. In general the problem is coNP-complete, and on the family built to force exact exploration the checker visited 262,141 states at depth 16, in 0.30 s. At run time, a decision costs about 0.8 μs of engine time per stage.

For recertification, the number of stale routes stayed nearly flat as random hierarchies grew, but it grew linearly with length in cascades, where a mean of 32.5 of 64 routes went stale (Table IX). A change affects only the routes that pass through it, so early stages in long chains are the costly ones. No locality violation was found in 94,840 checks.

This answer measures recertification by routes, not by time or new labels, so it does not show a cost saving over whole-policy recertification. The random hierarchies are also small, and the timing experiments E9 and E10 were not rerun after the last engine change, which added only constructor checks.

H. RQ4: Real Configurations and Published Designs

E15. We analyzed 138 example configurations from a single commit of the NeMo Guardrails repository, including Colang 2 flows and agent hook rails. Rails were classified by name using the rules recorded in the artifact. Of these configurations, 56 contain guardrails, using 78 distinct flows. The median configuration has 2 stages, and the largest has 9. The framework has no escalation construct: the rails within a stage form a chain in which any rail can stop the request. E15 describes structure only: we did not encode these configurations in HSL, so it yields no composition verdicts.

The rail classification identified 42 prompted LLM tasks, 60 other models or external services, 10 deterministic rails, and 8 rails that rewrite their inputs. The rewriting rails cannot be expressed in our language. Among the configurations, 47 are entirely statistical, 3 are entirely deterministic, and 4 guard tool calls. The deterministic configurations include regex rails for coding-agent hooks.

These examples are predominantly short chains of learned stages operating on text rather than actions. For such chains, the graph analysis is straightforward; the main output of interest is the list of assumptions. However, teaching examples from one framework do not establish the structure of production deployments. The corpus provides no evidence that escalation hierarchies are common, although these occur in published research designs and in DevAgent. No examined configuration provides facts in the sense of our calculus, so the corpus contains no observed instance of fact propagation to which the corresponding results in Section IV-E apply.

E13. We modeled published designs from their publications, applying the two uniform obligations defined in Section VII. One modeller produced all specifications, and the initial models were revised after inspecting results. Because the verdicts depend directly on the encoding rule, we treat this experiment as an exploratory illustration. A rejection indicates that the published description, interpreted under that rule, does not supply the assumptions needed to justify the composition; it does not establish that the deployed system is insecure. Table X reports verdicts under benign and adversarial threat models, together with the obligations supporting an adversarial acceptance or needed to obtain one.

NeMo Guardrails [5] and LlamaFirewall [4] report measurements for their learned stages rather than bounds for a particular deployment. Under our encoding rule, their goals therefore remain open even under the benign threat model. This finding does not negate their reported performance: LlamaFirewall, for example, reports reduced attack success on AgentDojo at a modest utility cost. Certifying the stages for a deployment would supply the missing obligation. Our analysis identifies the claims that such a deployment would need to support. NeMo focuses on conversational applications, and its authors recommend combining rail types.

CaMeL [15] passes under both threat models, conditional on its residual assumptions, including a trusted query and adequate policies. Our model assumes that its planner reads only the trusted query. Classifying that planner's view as raw would cause rejection, so the verdict depends on the design's stated isolation premise. The encoding also simplifies the design: planning generates proposals, and policy failures may lead to a user query.

Trust or Escalate [8] and Conformal Arbitrage [9] pass only under the benign threat model, which matches their intended setting. Neither is presented as a security mechanism. Trust or Escalate concerns the evaluation of model outputs, while Conformal Arbitrage uses one threshold and provides a whole-

TABLE X
DESIGNS AS MODELLED IN HSL: VERDICT PER THREAT MODEL, ROUTES, AND WHAT WOULD FLIP THE ADVERSARIAL VERDICT (EXPLORATORY)

Design	Benign	Adversarial	Routes	Obligations that the adversarial verdict rests on, or that would flip it
DevAgent	accepted	accepted	2	rests on: human_attentive, miss_rate_bounded, safe_on_this_text
DevAgent, exposed judge	accepted	rejected	1	supply: safe_on_this_text
LlamaFirewall	rejected	rejected	1	supply: miss_rate_bounded, safe_on_this_text
NeMo rails	rejected	rejected	1	supply: miss_rate_bounded, safe_on_this_text
CaMeL-style	accepted	accepted	1	rests on: memory_uncompromised, policy_adequate, trusted_query_and_interpreter
Trust or Escalate	accepted	rejected	3	supply: safe_on_this_text
Conformal Arbitrage	accepted	rejected	2	supply: safe_on_this_text

policy expected-loss guarantee. Our route model simplifies the latter guarantee.

I. Answer to RQ4

Public configurations are mostly short chains of statistical stages, and the framework makes explicit the assumptions that published designs would need to support. Of the 138 configurations examined, 56 contain guardrails, with a median of 2 stages and a maximum of 9. Of those, 47 are entirely statistical, 3 are entirely deterministic, and 4 guard tool calls. None has an escalation construct or provides facts, and 8 rails rewrite their inputs, which our language cannot express.

Under the uniform encoding rule, Table X shows three outcomes for the published designs. LlamaFirewall and NeMo Guardrails are rejected under both threat models until miss_rate_bounded and safe_on_this_text are supplied. Trust or Escalate and Conformal Arbitrage pass under the benign threat model only, which matches their intended setting. CaMeL passes under both, conditional on its residual assumptions, including a trusted query and adequate policies.

These findings are exploratory. They describe assumptions under one encoding rule applied by one modeller, and a rejection does not show that a deployed system is insecure. The corpus comes from teaching examples of a single framework, so it does not characterize production deployments, and it received no composition verdicts. No practitioner has assessed whether these outputs are useful.

J. Failures, Deviations and Continuity

Reviews of earlier versions identified defects documented in the artifact's errata alongside the original results. A reported measurement was a constant, and no experiment exercised the contract-satisfaction code. Fingerprints omitted checked predicates and closures, allowing stale certificates to remain open. The initial statement of Theorem 9 was false. Assumption lists omitted dependencies involving provided facts and veto stages whose only contribution was to provide facts. The initial corpus extraction missed the Colang 2 configurations. Finally, the PRISM model incorporated the checker's crediting rule, compromising its role as an alternative encoding and making it appear to suffer from state-space explosion. These defects were corrected and the experiments rerun.

An attempted proof of a generalized meet lemma also revealed that the lemma was false, leading to Theorem 6. Earlier E9 runs exposed quadratic growth in the number of goals; that implementation cost has since been removed. In particular, E12 failed the original literal decision rule on two laws in the first run, and the rule was relaxed afterward.

The artifact retains an earlier preregistered scripted study of a fixed pipeline. Its certificate requirement prevented deployment in all 10 primary seeds, exposing a floating-point defect in a scripted scorer. Any acceptance test based on joint risk would have detected the same issue. After the defect was corrected, certification changed no decisions. A simple provenance rule achieved lower risk than the full pipeline because the sandbox could not exercise semantic guards. Moreover, no semantic mechanism withstood a distribution shift associated with an assumption identified by the residual list. These results remain part of the record. They are consistent with the preference for deterministic establishment expressed by Theorem 4 and Table I, but provide no evidence of semantic guard effectiveness.

K. Threats to Validity

Internal. One author developed the calculus, checker, reference interpreter, encodings, and specifications, creating a risk of shared assumptions across the test oracles. Correspondence between the reference and the equations is checked only on sampled terms, and the script applying the induction schema is not itself verified. A proof goal could therefore be misstated and still be proved, although definition mutants and adequacy tests reduce this risk.

Construct. Safety is relative to the declared goals and contracts. Weak goals are easy to satisfy, false contracts invalidate the corresponding guarantees, and declaring an assumed obligation can silence a rejection without establishing that obligation. Safety also does not imply utility: the calculus accepts a hierarchy that denies nearly every proposal. Per-decision risk does not capture harm accumulated across a session, and a small joint-risk bound may reflect rare attacks rather than effective detection.

External. The language cannot directly express rewriting or voting, and the synthetic families isolate structural properties rather than represent typical deployments. Timings come from one machine and a Python implementation; PRISM's symbolic engines were unavailable. No practitioners participated in the evaluation.

Conclusion. The coverage estimates in E12 and E12b are consistent with nominal coverage within Monte Carlo

sampling error. However, E12b uses small finite probability laws generated by our own procedure. The fitted node-scaling slopes are close to the protocol's threshold, so Theorem 6, rather than the empirical fits alone, supports the claim of near-linear complexity.

XI. Related Work

Compositional verification in software engineering. Our checker is modular verification over a loop-free program, in the tradition of verification conditions for passive programs [47], with atomic facts in place of formulas, and its core is available-expressions analysis [34], [35]. What is specific is the domain model: adversarial stages, conditional contracts and evidence scoped to an invocation. COVERT also finds composition failures that no single component shows, by analyzing inter-app permission leakage compositionally [46]. Chaki et al. verify across intellectual-property boundaries, where components are known only by their promises [48], a situation close to that of third-party guards. He et al. learn assumptions for compositional verification of probabilistic models [49]. Learning obligations rather than declaring them is a natural direction for us. Su et al. bound how model checking results move with estimated parameters [50], which relates to drift. Differential validation of executable semantics [44] is our model for conformance. Incremental recertification resembles build caching and regression test selection, and incremental product-line verification [51]; prefix fingerprints are an application of that idea to statistical evidence.

Contracts. Assume-guarantee reasoning [10], [11], [31] and contract-based design [29], [30] are our foundations, and we use a small fragment. Stochastic contracts [32], [33] give richer algebras. A position paper proposes layers whose probabilistic guarantee discharges the next layer's assumption, with system bounds by the chain rule [17]. We take a different route: learned stages are measured at the level of a whole route, on the deployed function, so that no probabilistic guarantee is composed. This gives up per-layer modularity for statistical goals and avoids the conditional-independence questions of chaining. Work on redundancy and diversity in security [52] treats the related question of what independent layers buy.

Runtime enforcement for agents. AgentSpec specifies runtime rules [7], ContrAgent compiles assume-guarantee temporal contracts into monitors [13], and behavioral contracts [14], ShieldAgent [53], VeriGuard [54], ProbGuard [55] and NaSh [56] enforce or verify agent policies. These handle richer temporal properties of agent behavior, and several run on real agents, which we do not. Our question is the joint meaning of a stack of mechanisms. Shielding [57], [58] and the Simplex architecture [59] are ancestors of the monitor-first design.

Provenance and isolation. CaMeL [15], Fides [16] and the dual-LLM pattern [23] give guarantees by construction, on real benchmarks in the first two cases. Our calculus expresses their structure, and a provided fact is the analogue of what they achieve, a deterministic mechanism establishing a property that later stages can use, although our model of CaMeL does not use one.

TABLE XI
Positioning: Y Yes, P Partial, N No or Not Stated, – Not a Goal

Approach	Guards	Attacks	Temp.	Check	Stat.	Exact
LlamaFirewall [4]	Y	Y	P	N	N	–
CaMeL [15]	Y	Y	P	P	–	–
AgentSpec [7]	Y	Y	Y	–	N	–
ContrAgent [13]	P	P	Y	P	N	N
Trust or Escalate [8]	Y	–	–	P	Y	–
Layered A/G [17]	N	N	P	P	Y	N
Tint	N	N	N	Y	Y	Y

Guards: evaluated with real guards. Attacks: with real attacks. Temp.: temporal properties. Check: composition check. Exact: a completeness result. ContrAgent and the layered paper were rated from their abstracts.

Cascades and statistical guarantees. Trust or Escalate [8] and Conformal Arbitrage [9] calibrate specific cascades, conformal methods control loss for LLM outputs [60]–[62], and model cascades trade cost and quality [63]. These supply evidence for a route. ScenicProver composes probabilistic evidence for learning-enabled systems [64].

Guardrail frameworks and benchmarks. LlamaFirewall [4] and NeMo Guardrails [5] are the systems whose configurations and designs we analyze. Benchmarks [3], [65], [66] measure effectiveness, which our results do not address and do not replace. Table XI positions the closest work, as rated by the author from primary sources. The first two columns are where others lead.

XII. Conclusion

In this study, we introduced a formal framework for specifying, verifying, and assessing the safety of composed guardrail hierarchies in large language model (LLM) agents. By modeling guardrails through explicit contracts and conditional evidence, our approach establishes when the guarantees of individual stages are sufficient to justify the safety requirements of the overall hierarchy, while identifying unsupported assumptions and providing counterexamples for invalid compositions. We established soundness and relative completeness of the composition check, characterized its computational complexity, and extended the framework with route-based risk assessment and incremental recertification to support the reassessment of modified configurations. The results support the correctness of the verification procedure within a stated trusted base, confirm near-linear checking for hierarchies without fact propagation, although the general problem is coNP-complete, and show that recertification scope depends more on a hierarchy's structure than on its size. An exploratory analysis of published designs illustrates how the framework can expose assumptions that are not apparent from evaluating individual guardrails in isolation. However, the framework establishes conditional guarantees about composition rather than the correctness of the underlying guardrails, the public configurations we examined received no composition verdicts, and its practical effectiveness in supporting real-world deployment decisions remains to be investigated. Overall, this work provides a formal foundation for making the assumptions, dependencies, and limitations underlying LLM guardrail safety claims explicit, auditable, and amenable to systematic verification.

Data Availability

The project repository [67] contains the verification library, the HSL checker and execution engine, proof obligations, experimental protocol, evaluation drivers, raw results, errata, and manuscript source. The accompanying `ARTIFACT.md` maps the paper's claims to reproducible commands and their execution times. All proofs can be executed in under one minute on a laptop CPU, without requiring a GPU or API key. The evaluation corpus includes only the guardrail-related sections of publicly available configurations, together with their source commit identifiers.

Declaration on the Use of AI

Generative AI tools were used to assist with organizing the repository code and checking the manuscript for grammatical errors. The author reviewed the generated suggestions and takes full responsibility for the accuracy and integrity of all claims, proofs, references, and the final manuscript.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-world Github Issues?" in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 54 107–54 157.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM workshop on artificial intelligence and security*, 2023, pp. 79–90.

[3] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," *Advances in neural information processing systems*, vol. 37, pp. 82 895–82 920, 2024.

[4] S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding, S. Wan, S. Whitman, L. Deason, N. Doucette, A. Montilla *et al.*, "LlamaFirewall: An open source guardrail system for building secure AI agents," *arXiv preprint arXiv:2505.03574*, 2025.

[5] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2023, pp. 431–445.

[6] M. Shamsujjoha, Q. Lu, D. Zhao, and L. Zhu, "Swiss Cheese Model for AI Safety: A Taxonomy and Reference Architecture for Multi-Layered Guardrails of Foundation Model Based Agents," in *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*. IEEE, 2025, pp. 37–48.

[7] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*. ACM, 2026, pp. 2938–2950.

[8] J. Jung, F. Brahman, and Y. Choi, "Trust or Escalate: LLM Judges with Provable Guarantees for Human Agreement," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 3101–3125.

[9] W. Overman and M. Bayati, "Conformal Arbitrage: Risk-Controlled Balancing of Competing Objectives in Language Models," *Advances in Neural Information Processing Systems*, vol. 38, pp. 70 305–70 337, 2026.

[10] J. Misra and K. M. Chandy, "Proofs of networks of processes," *IEEE Trans. Softw. Eng.*, vol. 7, no. 4, p. 417–426, Jul. 1981.

[11] M. Abadi and L. Lamport, "Composing specifications," *ACM Trans. Program. Lang. Syst.*, vol. 15, no. 1, p. 73–132, Jan. 1993.

[12] B. Christianson and W. S. Harbison, "Why isn't trust transitive?" in *Security Protocols Workshop*, 1996.

[13] Y. Xiao and P. Nuzzo, "Symbolic Temporal Supervision of LLM Agents Using Contracts," *arXiv preprint arXiv:2609.18128*, 2026.

[14] V. P. Bhardwaj, "Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents," arXiv:2602.22302, 2026.

[15] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating Prompt Injections by Design," in *2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 2026, pp. 587–618.

[16] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI Agents with Information-Flow Control," arXiv:2505.23643, 2025.

[17] S. Bensalem, Y. Dong, M. Franzle, X. Huang, J. Kroger, D. Nickovic, A. Nouri, R. Roy, and C. Wu, "Position: A Three-Layer Probabilistic Assume-Guarantee Architecture Is Structurally Required for Safe LLM Agent Deployment," arXiv:2605.18672, 2026.

[18] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations," arXiv:2312.06674, 2023.

[19] S. Han, K. Rao, A. Ettinger, L. Jiang, B. Y. Lin, N. Lambert, Y. Choi, and N. Dziri, "WildGuard: Open One-stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs," in *Advances in Neural Information Processing Systems*, vol. 37. Curran Associates, Inc., 2024, pp. 8093–8131.

[20] M. Sharma, M. Tong, J. Mu, J. Wei, J. Kruthoff, S. Goodfriend, E. Ong, A. Peng, R. Agarwal, C. Anil *et al.*, "Constitutional Classifiers: Defending against Universal Jailbreaks across Thousands of Hours of Red Teaming," arXiv:2501.18837, 2025.

[21] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2025, pp. 29 680–29 697.

[22] W. Hackett, L. Birch, S. Trawicki, N. Suri, and P. Garraghan, "Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems," in *Proceedings of the First Workshop on LLM Security (LLMSEC)*. Association for Computational Linguistics, 2025, pp. 101–114.

[23] S. Willison, "The dual LLM pattern for building AI assistants that can resist prompt injection," Blog post, Apr. 2023.

[24] J. P. Anderson, "Computer security technology planning study," U.S. Air Force Electronic Systems Division, Tech. Rep. ESD-TR-73-51, 1972.

[25] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[26] F. B. Schneider, "Enforceable security policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000.

[27] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.

[28] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 1, pp. 5–19, 2003.

[29] B. Meyer, "Applying design by contract," *Computer*, vol. 25, no. 10, pp. 40–51, 1992.

[30] A. Benveniste, B. Caillaud, D. Nickovic, R. Passerone, J.-B. Raclet, P. Reinkemeier, A. Sangiovanni-Vincentelli, W. Damm, T. Henzinger, and K. G. Larsen, "Contracts for system design," Ph.D. dissertation, Inria, 2012.

[31] C. B. Jones, "Tentative steps toward a development method for interfering programs," *ACM Transactions on Programming Languages and Systems*, vol. 5, no. 4, pp. 596–619, 1983.

[32] B. Delahaye, B. Caillaud, and A. Legay, "Probabilistic contracts: A compositional reasoning methodology for the design of systems with stochastic and/or non-deterministic aspects," *Formal Methods in System Design*, vol. 38, no. 1, pp. 1–32, 2011.

[33] P. Nuzzo, J. Li, A. L. Sangiovanni-Vincentelli, Y. Xi, and D. Li, "Stochastic assume-guarantee contracts for cyber-physical system design," *ACM Transactions on Embedded Computing Systems*, vol. 18, no. 1, pp. 1–26, 2019.

[34] G. A. Kildall, "A unified approach to global program optimization," in *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '73)*. ACM, 1973, pp. 194–206.

[35] J. B. Kam and J. D. Ullman, "Monotone data flow analysis frameworks," *Acta Informatica*, vol. 7, no. 3, pp. 305–317, 1977.

[36] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[37] H. Bu and M. Sun, "Clopper-pearson algorithms for efficient statistical model checking estimation," *IEEE Transactions on Software Engineering*, vol. 50, no. 7, pp. 1726–1746, 2024.

[38] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei, "Learn then test: Calibrating predictive algorithms to achieve risk control," *The Annals of Applied Statistics*, vol. 19, no. 2, pp. 1641–1662, 2025.

[39] A. N. Angelopoulos and S. Bates, "Conformal Prediction: A Gentle Introduction," *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023.

[40] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," in *Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, vol. 4963. Springer, 2008, pp. 337–340.

[41] H. Barbosa, C. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli, and Y. Zohar, "cvc5: A Versatile and Industrial-Strength SMT Solver," in *Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, vol. 13243. Springer, 2022, pp. 415–442.

[42] D. Jackson, "Alloy: A Language and Tool for Exploring Software Designs," *Communications of the ACM*, vol. 62, no. 9, pp. 66–76, 2019.

[43] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[44] K. Wang, J. Wang, C. M. Poskitt, X. Chen, J. Sun, and P. Cheng, "K-st: A formal executable semantics of the structured text language for plcs," *IEEE Transactions on Software Engineering*, vol. 49, no. 10, pp. 4796–4813, 2023.

[45] M. Kwiatkowska, G. Norman, and D. Parker, "PRISM 4.0: Verification of probabilistic real-time systems," in *International conference on computer aided verification*. Springer, 2011, pp. 585–591.

[46] H. Bagheri, A. Sadeghi, J. Garcia, and S. Malek, "COVERT: Compositional Analysis of Android Inter-App Permission Leakage," *IEEE Transactions on Software Engineering*, vol. 41, no. 9, pp. 866–886, 2015.

[47] M. Barnett and K. R. M. Leino, "Weakest-precondition of unstructured programs," in *Proceedings of the 6th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering*. ACM, 2005, pp. 82–87.

[48] S. Chaki, C. Schallhart, and H. Veith, "Verification across intellectual property boundaries," *ACM Transactions on Software Engineering and Methodology*, vol. 22, no. 2, pp. 1–12, 2013.

[49] F. He, X. Gao, M. Wang, B.-Y. Wang, and L. Zhang, "Learning weighted assumptions for compositional verification of markov decision processes," *ACM Transactions on Software Engineering and Methodology*, vol. 25, no. 3, pp. 1–39, 2016.

[50] G. Su, Y. Feng, T. Chen, and D. S. Rosenblum, "Asymptotic perturbation bounds for probabilistic model checking with empirically determined probability parameters," *IEEE Transactions on Software Engineering*, vol. 42, no. 7, pp. 623–639, 2015.

[51] C. Kröher, M. Flöter, L. Gerling, and K. Schmid, "Incremental software product line verification-a performance analysis with dead variable code," *Empirical Software Engineering*, vol. 27, no. 3, p. 68, 2022.

[52] B. Littlewood and L. Strigini, "Redundancy and diversity in security," in *Computer Security – ESORICS 2004*, ser. Lecture Notes in Computer Science, vol. 3193. Springer, 2004, pp. 423–438.

[53] Z. Chen, M. Kang, and B. Li, "Shieldagent: Shielding agents via verifiable safety policy reasoning," *arXiv preprint arXiv:2503.22738*, 2025.

[54] L. Miculicich, M. Parmar, H. Palangi, D. Dvijotham, M. Montanari, T. Pfister, and L. Le, "VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation," in *ACL 2026*, 2026.

[55] H. Wang, C. M. Poskitt, J. Wei, and J. Sun, "ProbGuard: Proactive Runtime Monitoring for LLM Agent Safety via Probabilistic Prediction," in *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2026, pp. 1470–1482.

[56] B. R. Gyawali, S. Achalla, K. Kallas, and S. Kumar, "NaSh: Guardrails for an LLM-Powered Natural Language Shell," arXiv:2506.13028, 2025.

[57] R. Bloem, B. Könighofer, R. Könighofer, and C. Wang, "Shield Synthesis: Runtime Enforcement for Reactive Systems," in *Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, vol. 9035. Springer Berlin Heidelberg, 2015, pp. 533–548.

[58] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe Reinforcement Learning via Shielding," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[59] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, p. 20, 2001.

[60] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, "Conformal Risk Control," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 55 198–55 218.

[61] C. Y.-C. Chen, J. Shen, Z. Deng, and L. Lei, "Conformal tail risk control for large language model alignment," *arXiv preprint arXiv:2502.20285*, 2025.

[62] C. Mohri and T. Hashimoto, "Language models with conformal factuality guarantees," *arXiv preprint arXiv:2402.10978*, 2024.

[63] L. Chen, M. Zaharia, and J. Zou, "Frugalgpt: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023.

[64] E. Vin, K. A. Miller, I. Incer, S. A. Seshia, and D. J. Fremont, "ScenicProver: A Framework for Compositional Probabilistic Verification of Learning-Enabled Systems," arXiv:2511.02164, 2025.

[65] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 35 331–35 366.

[66] M. Andriushchenko, A. Souly, M. Dziemian, D. Duenas, M. Lin, J. Wang, D. Hendrycks, A. Zou, Z. Kolter, M. Fredrikson, Y. Gal, and X. Davies, "AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 79 185–79 220.

[67] M. Babaei, "trust-is-not-transitive," https://github.com/ai4se4ai-lab/trust-is-not-transitive, 2026, GitHub repository.